

Lógica y Representación 1

Ciclos

Ingeniería de Sistemas
Universidad de Antioquia

Método de Polya - Resumen



PROBLEMA



ALGORITMO



1

Entender el problema

2

Diseñar un plan

3

Implementar el plan

4

Revisar el plan

Entrada, procesamiento y salida

Los programas de computadora realizan estos tres pasos:

1. Se recibe la **entrada**.
2. Se realiza un **proceso** sobre la entrada.
3. Se genera la **salida**



Inputs



Process

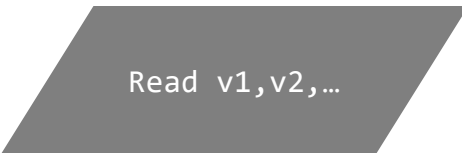
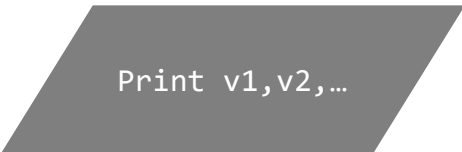


Outputs



Tabla resumen

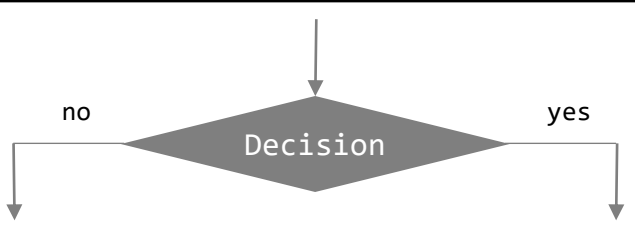
La siguiente tabla muestra la representación equivalente de las piezas esenciales para construir código secuencial

Tipo	Diagrama de bloques	Pseudocodigo	Python
Entrada 		Leer(v1,v2,...)	<pre>v1 = input(...) v2 = input(...)</pre>
Salida 		Escribir(v1,v2,...)	<pre>print(v1,v2,...)</pre>
Proceso 		$c = a + b$	$c = a + b$

En Python, `input()` siempre devuelve texto (`str`); si el valor se va a usar en una operación numérica, debe convertirse con `int()` o `float()` (por ejemplo: `v1 = float(input(...))`).

Tabla resumen

La siguiente tabla, agrega la nueva el nuevo bloque que permite implementar toma de decisiones en los programas.

Diagrama de bloques	Pseudocodigo	Python
	Si (condicion) Entonces instrucciones verdad Sino instrucciones falsa Fin_Si	<pre>if (condicion): instrucciones verdad else: instrucciones falso</pre>

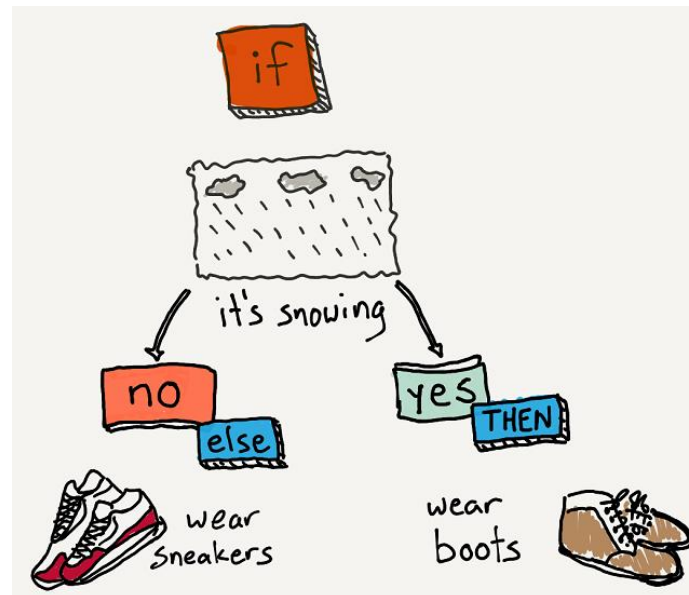


Imagen tomada de CS Intro ([link](#))

Casos en los problemas de decisión lógica

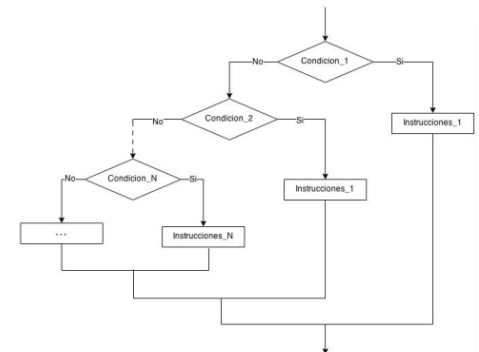
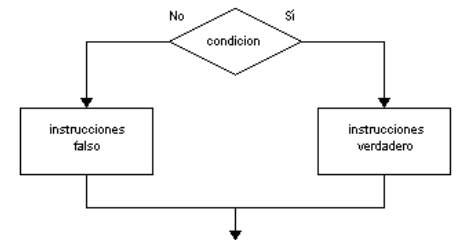
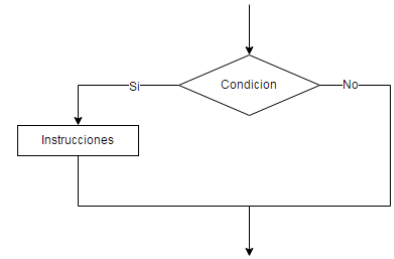


**Alternativas en los
problemas de decisión
lógica**

Alternativa simple

Alternativa doble

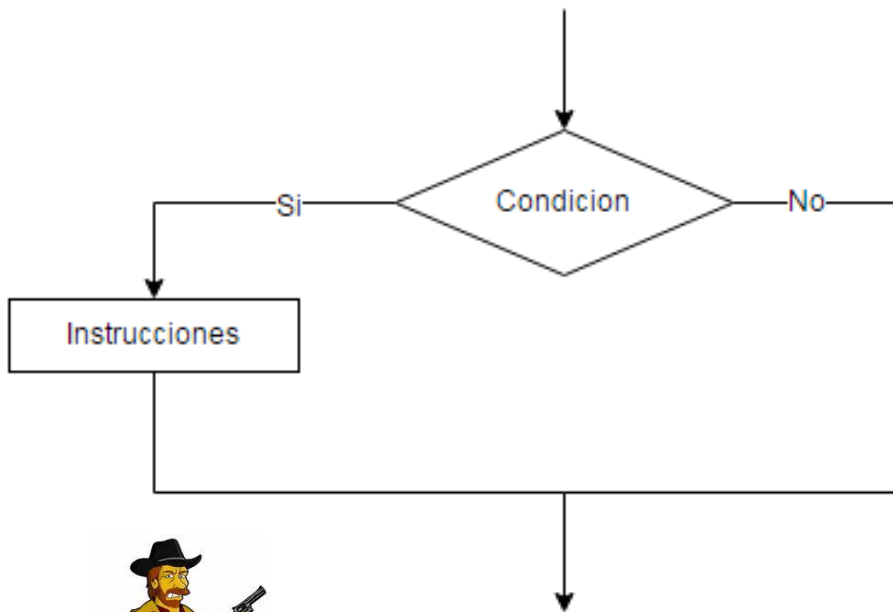
Alternativa múltiple



Alternativa simple

En los problemas de este tipo la condición **falsa** no posee instrucciones.

Diagrama de flujo



Pseudocódigo

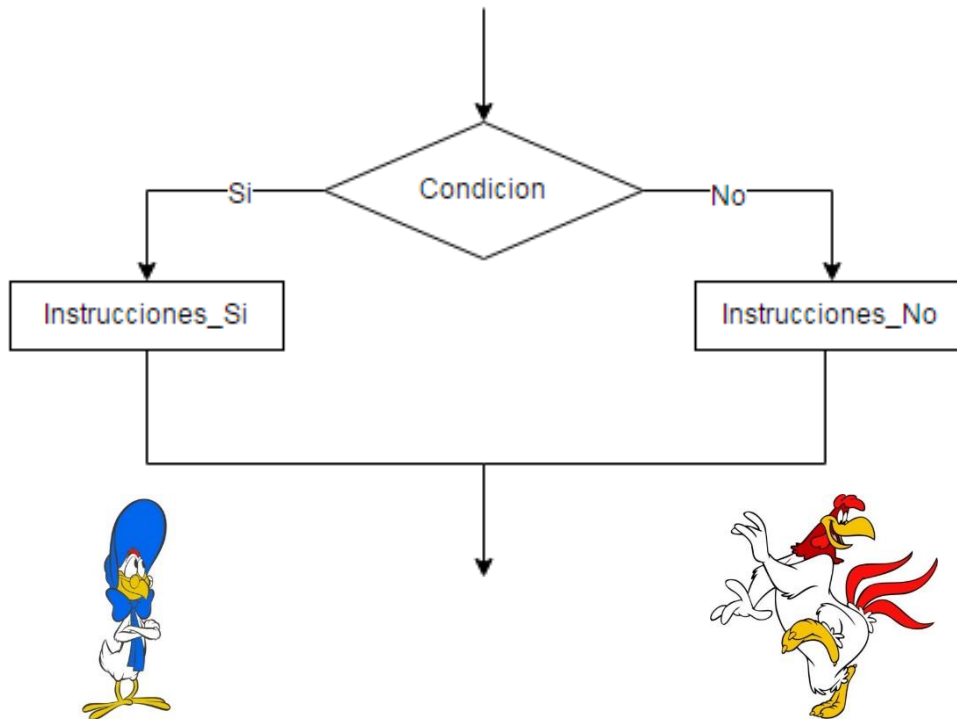
```
Si (condicion) Entonces  
    instrucciones condicion verdadera  
Fin_Si
```

Python

```
if (condicion):  
    instrucciones condicion verdadera
```

Alternativa doble

En los problemas de este tipo, ambos casos (**Falso** y **Verdadero**) poseen instrucciones asociadas.



Pseudocódigo

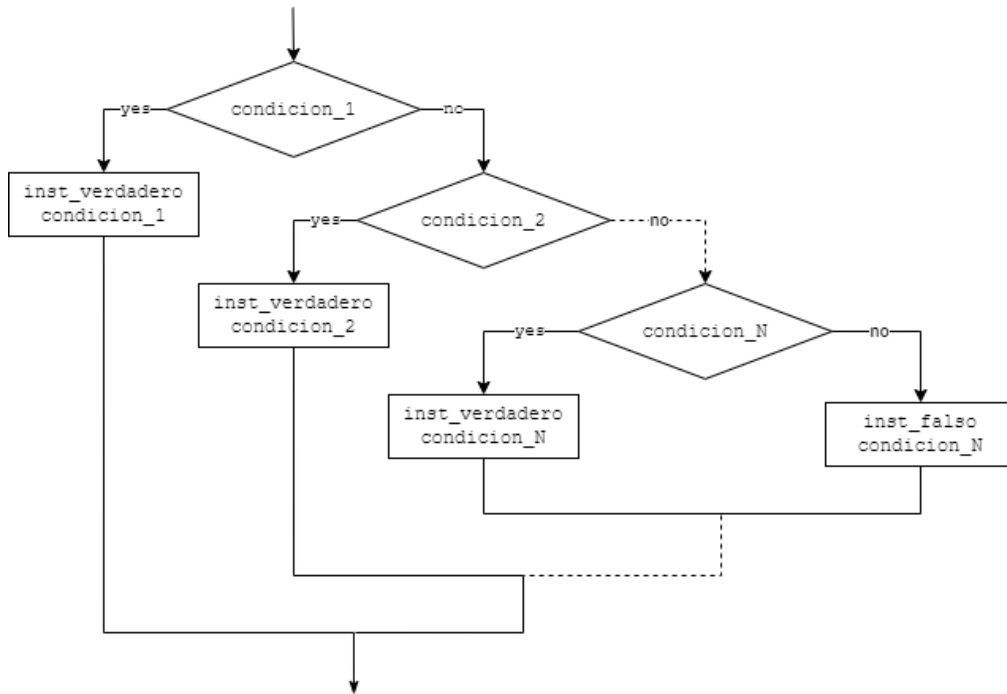
```
Si (condicion) Entonces  
    instrucciones condicion verdadera  
Sino  
    instrucciones condición falsa  
Fin_Si
```

Python

```
if (condicion):  
    instrucciones condicion verdadera  
else:  
    instrucciones condición falsa
```

Alternativa múltiple – Mismo criterio

Diagrama de flujo



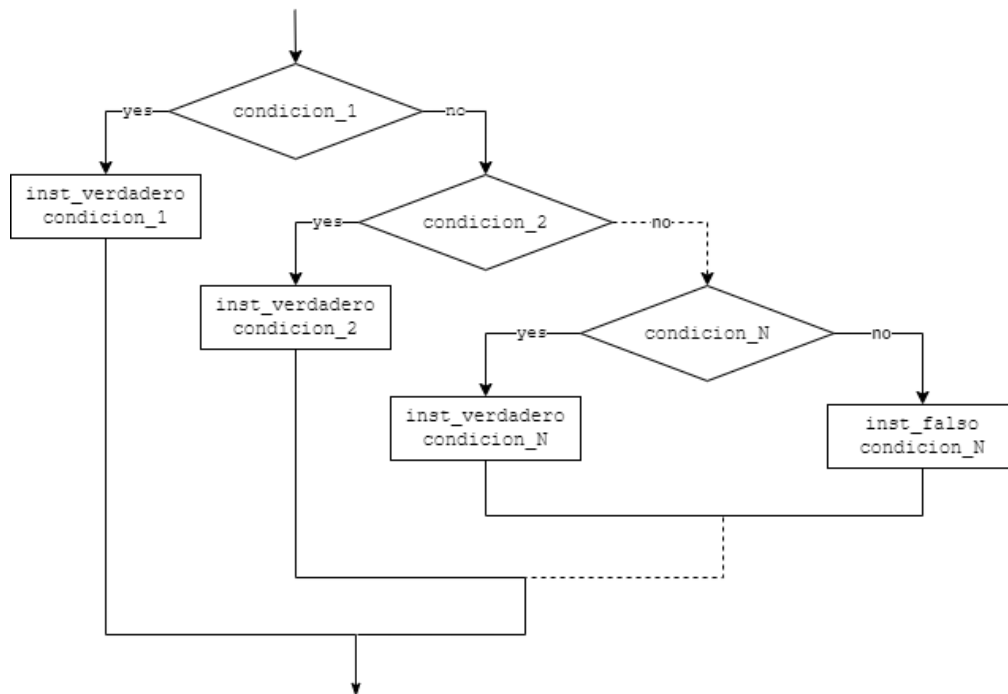
Pseudocódigo

```
Si (condicion_1) Entonces  
    condicion_1 verdad  
Sino  
    Si (condicion_2) Entonces  
        condicion_2 verdad  
    Sino  
        Si (condicion_3) Entonces  
            condicion_3 verdad  
        Sino  
            ...  
            Si (condicion_n) Entonces  
                condicion_n verdad  
            Sino  
                condicion_n falso  
            Fin_Si  
            ...  
        Fin_Si  
    Fin_Si  
Fin_Si
```

Alternativa múltiple – Mismo criterio

python

Diagrama de flujo

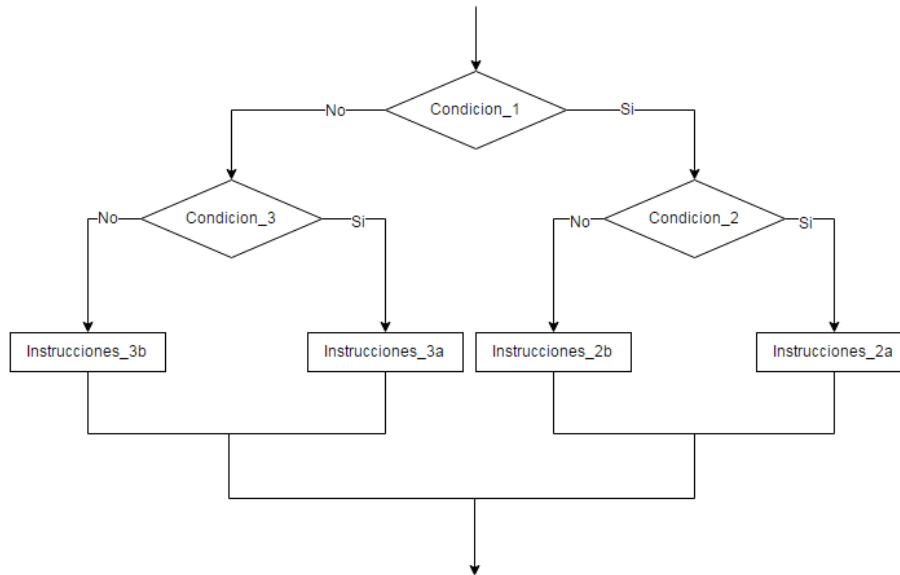


```
if (condicion_1):  
    condicion_1 verdad  
else:  
    if (condicion_2):  
        condicion_2 verdad  
    else:  
        if (condicion_3):  
            condicion_3 verdad  
        else:  
            ...  
            if (condicion_n):  
                condicion_n verdad  
            else:  
                condicion_n falso
```

```
if (condicion_1):  
    condicion_1 verdad  
elif (condicion_2):  
    condicion_2 verdad  
elif (condicion_3):  
    condicion_3 verdad  
...  
elif (condicion_n):  
    condicion_n verdad  
else:  
    condicion_n falso
```

Alternativa múltiple – Diferente criterio

Diagrama de flujo

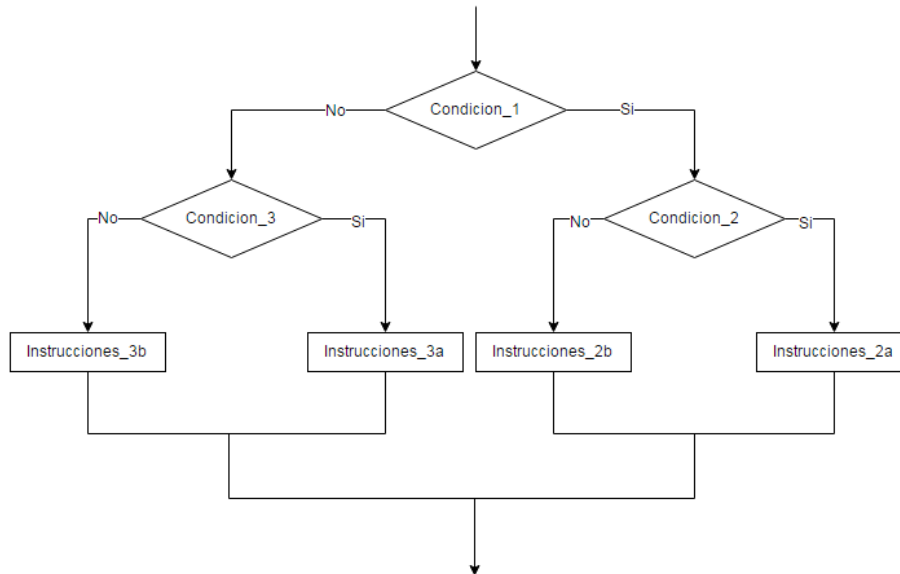


Pseudocódigo

```
Si (condicion_1) Entonces
    Si (condicion_3) Entonces
        condicion_3a verdad
    Sino
        condicion_3b
    Fin_Si
Sino
    Si (condicion_2) Entonces
        condicion_2a
    Sino
        condicion_2b
    Fin_Si
Fin_Si
```

Alternativa múltiple – Diferente criterio

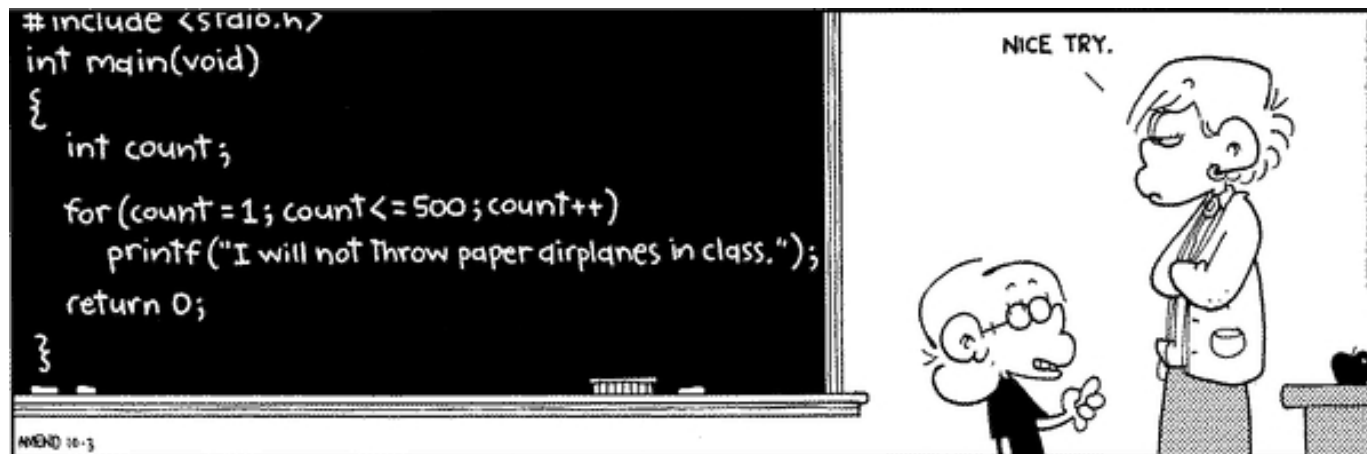
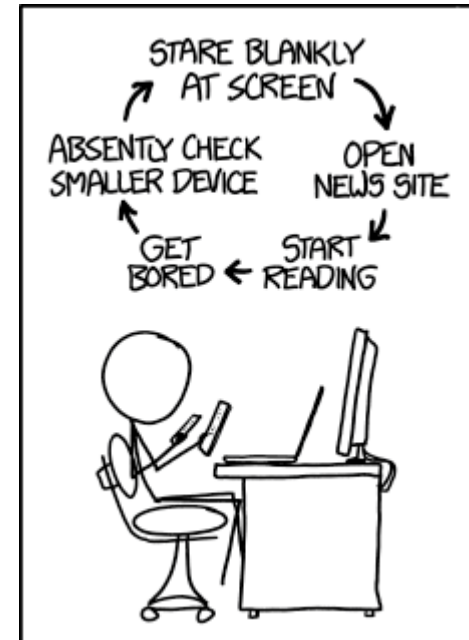
Diagrama de flujo



Python

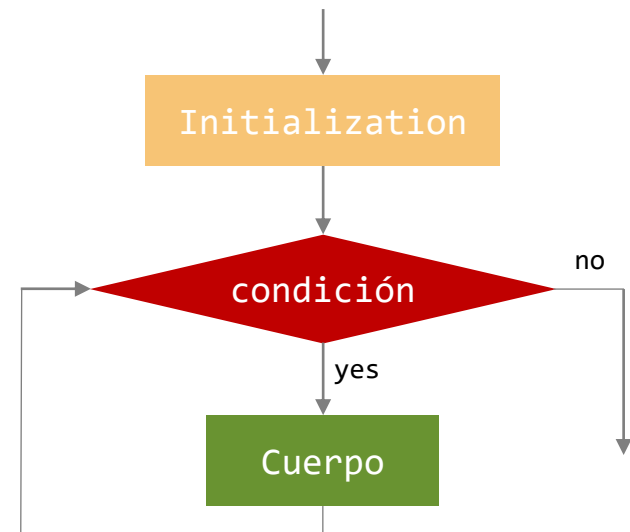
```
if (condicion_1):  
    if (condicion_3):  
        condicion_3a verdad  
    else:  
        condicion_3b  
else:  
    if (condicion_2):  
        condicion_2a  
    else:  
        condicion_2b
```

Ciclos



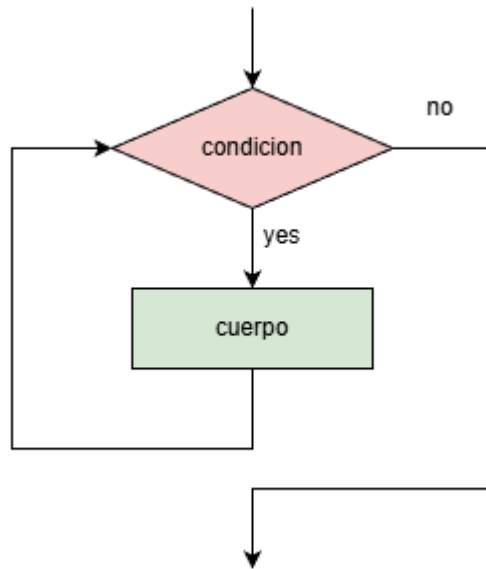
Ciclos

- Un **ciclo** (bucle o iteración) es una estructura que repite un conjunto de instrucciones mientras se cumpla una condición.
- Para describir un ciclo se requieren los siguientes componentes :
 - **Inicialización:** parte donde se establece el valor inicial de la variable de control, antes de que el ciclo arranque.
 - **Condición de control o parada:** Expresión lógica evaluada en cada iteración. Si es verdadera, el ciclo continúa; si es falsa, se detiene
 - **Cuerpo del ciclo:** Instrucciones que se ejecutan repetidamente. Incluye la modificación obligatoria de la variable de control para asegurar que el ciclo pueda terminar.



Ciclo mientras

Diagrama de flujo



Pseudocódigo

```
Mientras (condicion) Haga  
    instrucciones cuerpo  
Fin_Mientras
```

Python

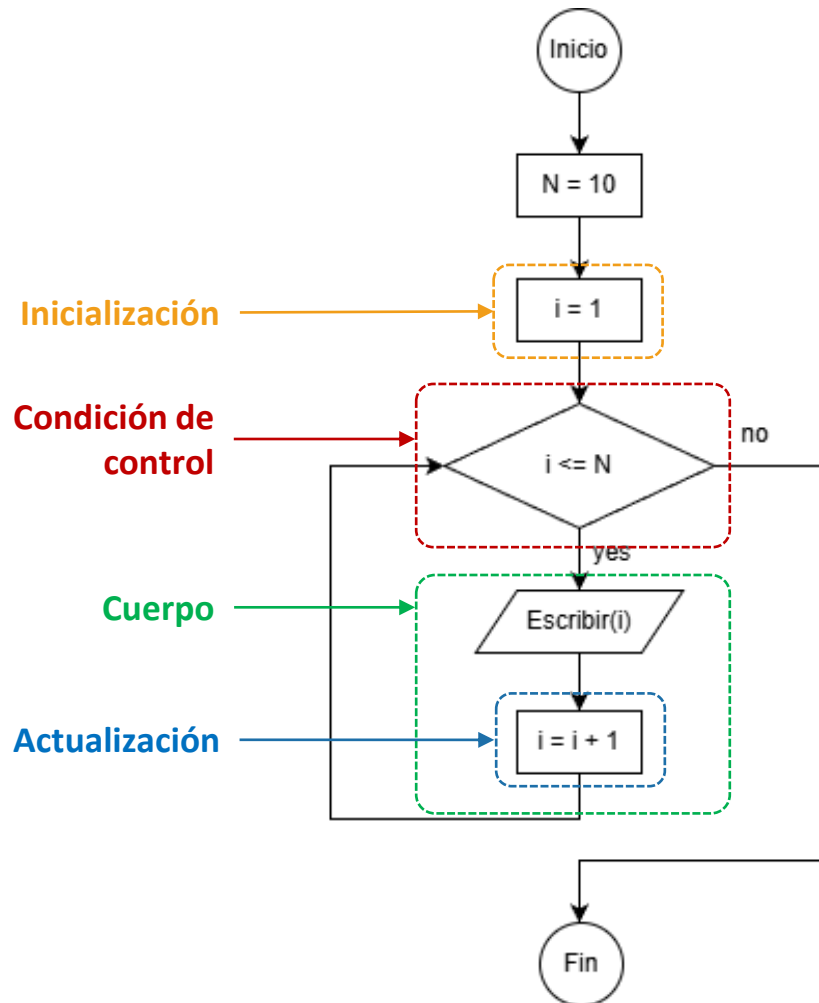
```
while (condicion):  
    instrucciones cuerpo
```

Observaciones importantes:

- Note que aunque la inicialización es importante para el ciclo, es previa a este (por eso no aparece).
- Dentro del cuerpo del ciclo están las instrucciones propias de lo que en este se debe realizar y la actualización de la variable de condición

Ciclos - Ejemplo 1

Retomando el problema inicial de imprimir los $N = 10$ primeros números, el uso de ciclos permite solucionar fácilmente la cantidad de números a imprimir cambiando solo el valor de N .



Pseudocódigo

```
Inicio
  N = 10
  i = 1
  Mientras (i <= N) Haga
    Escribir(i)
    i = i + 1
  Fin_Mientras
Fin
```

Python

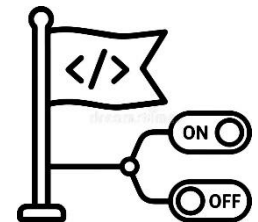
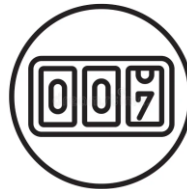
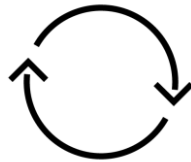
```
N = 10
i = 1
while i <= N:
    print(i)
    i = i + 1
```

[Link de ejecución](#)

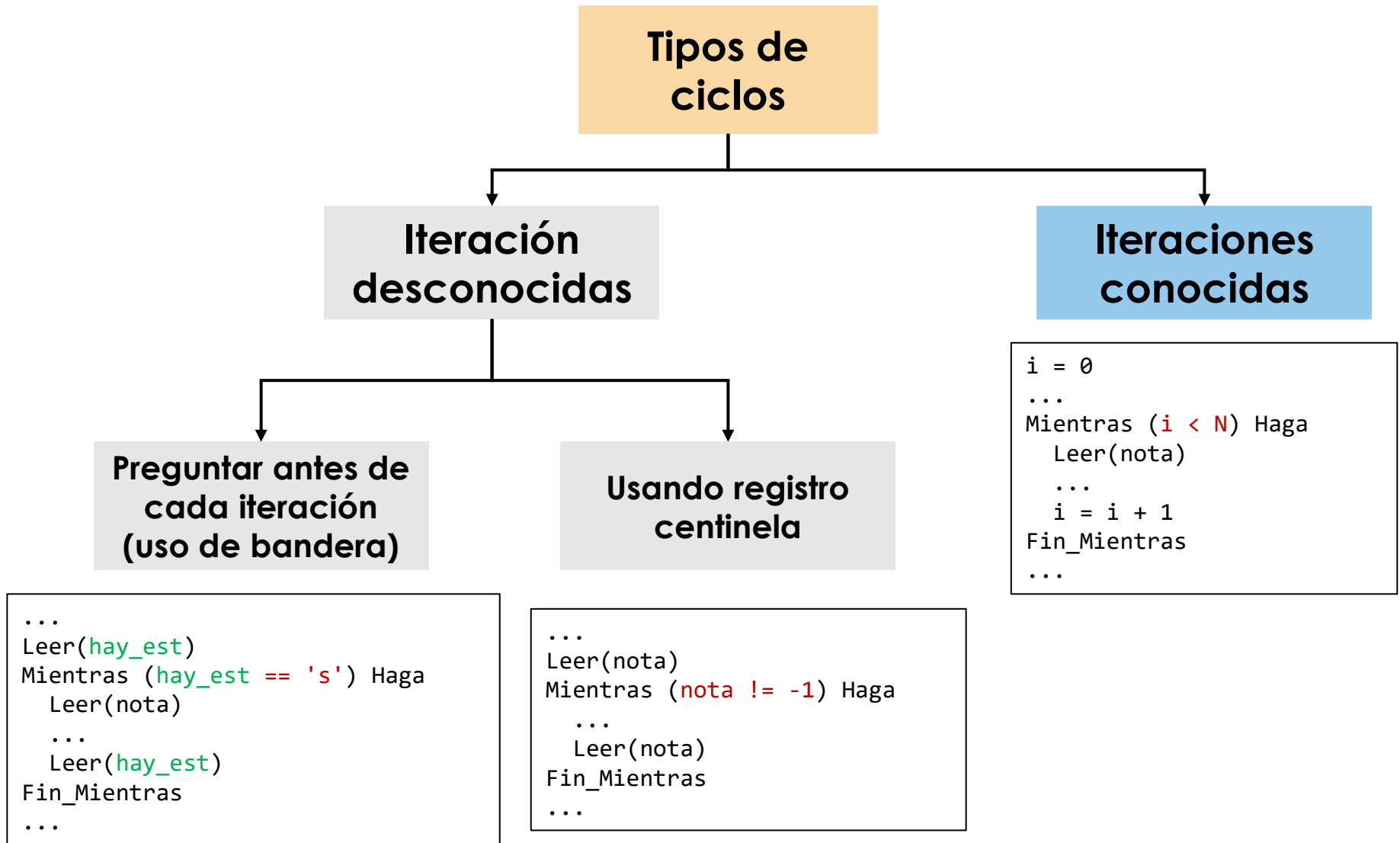
Tipos de variables útiles en los ciclos

Todo ciclo suele apoyarse en algunos tipos de variables que cumplen roles específicos:

- **Variable de control:** se evalúa en la condición del ciclo para decidir si continúa o termina
- **Contador:** lleva la cuenta de repeticiones; se actualiza en una cantidad constante.
- **Acumulador:** acumula el resultado de una operación que se repite; se actualiza en un valor variable.
- **Bandera:** variable de dos valores excluyentes usada para controlar el ciclo o recordar un estado dentro de él.
- **Centinela:** valor especial, fuera del rango válido de los datos, que señala el fin de la entrada.



Recapitulando lo visto



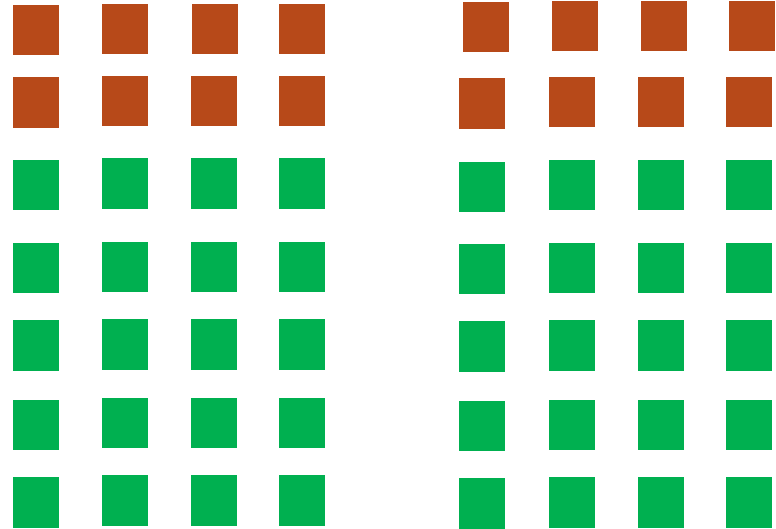
Ciclos – Problema repaso

El señor Burns, como buen capitalista que es, decidió construir un cine en Springfield. El cine es pequeño: tiene capacidad para 8 puestos VIP (a \$7000 cada uno) y 40 puestos normales (a \$5000 cada uno).

Para cada función, el señor Burns no sabe de antemano cuántos clientes van a asistir — puede que se llenen todos los puestos, o que la venta se cierre antes con puestos disponibles. Desarrolle una aplicación que:

- Registre la venta de una boleta solicitando al cajero el tipo de puesto (VIP o normal), y repita este registro hasta que ocurra una de estas dos condiciones: se agoten los 48 puestos, o el cajero ingrese -1 como tipo de puesto para indicar que no hay más clientes y cerrar la venta.
- Si el tipo de puesto solicitado ya alcanzó su capacidad máxima, no contabilizar esa venta, informarlo al cajero, y volver a solicitar el tipo de puesto (sin que esto se interprete como el cierre de la venta)
- Obtener, para cada tipo de puesto, el total de asistentes y la ganancia obtenida.
- Cuando se cierre la venta de boletas para la función (por cualquiera de las dos condiciones anteriores), imprimir el total de asistentes y las siguientes estadísticas:
 - El porcentaje de ocupación de cada zona (qué porcentaje de la capacidad VIP y de la capacidad Normal se logró vender).
 - El dinero total recaudado por las entradas VIP, por las entradas Normales y la taquilla total del cine.

Ciclos – Problema repaso



VIP: \$7000

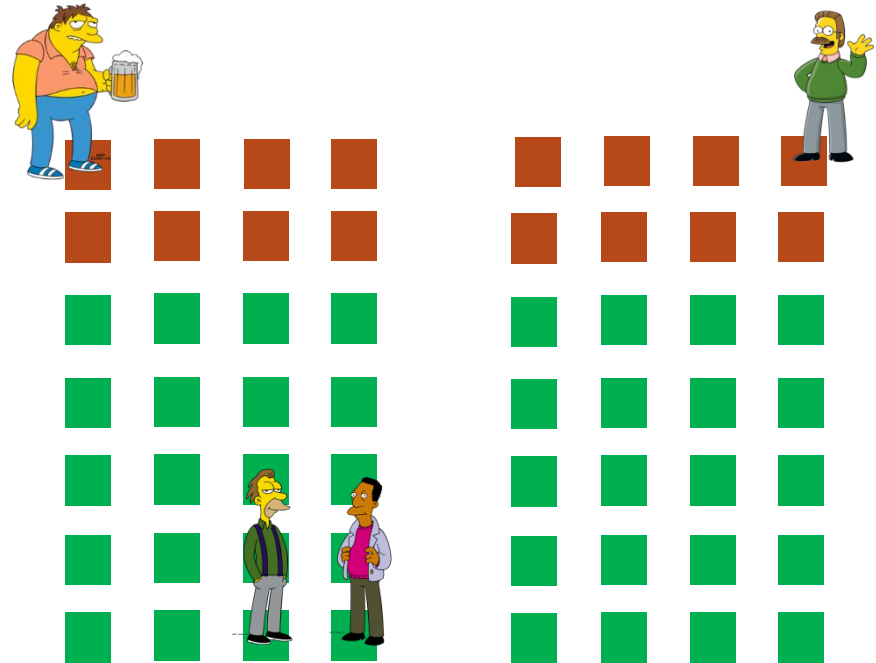


Normal: \$5000

Ciclos – Problema repaso



Puestos VIP	2
Puestos Normal	2
Total VIP	14000
Total Normal	10000
Asistentes	4
Total	24000



VIP: \$7000

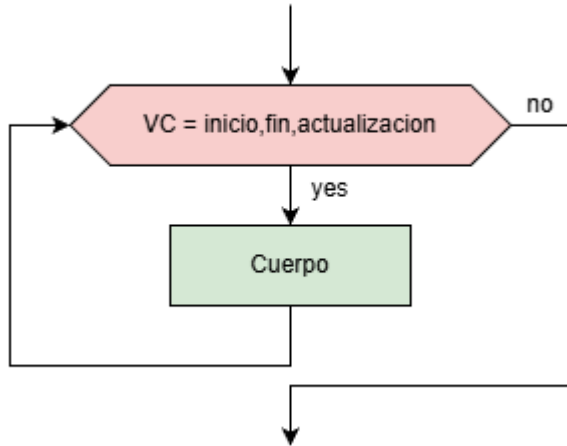


Normal: \$5000

Solución: [Link](#)

Ciclo Para

Diagrama de flujo



Pseudocódigo

```
Para (vc = inicio, fin, actualizacion) Haga  
    instrucciones cuerpo  
Fin_Para
```

Python

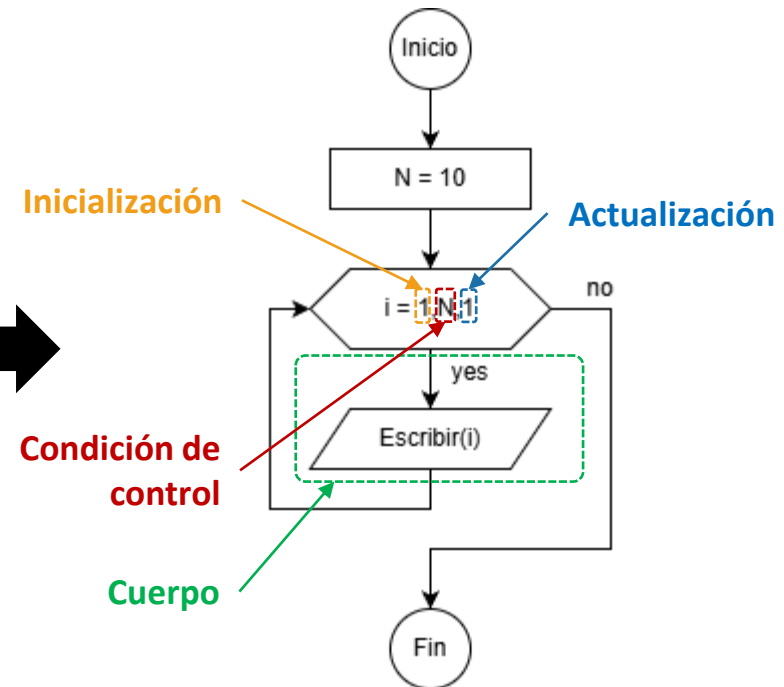
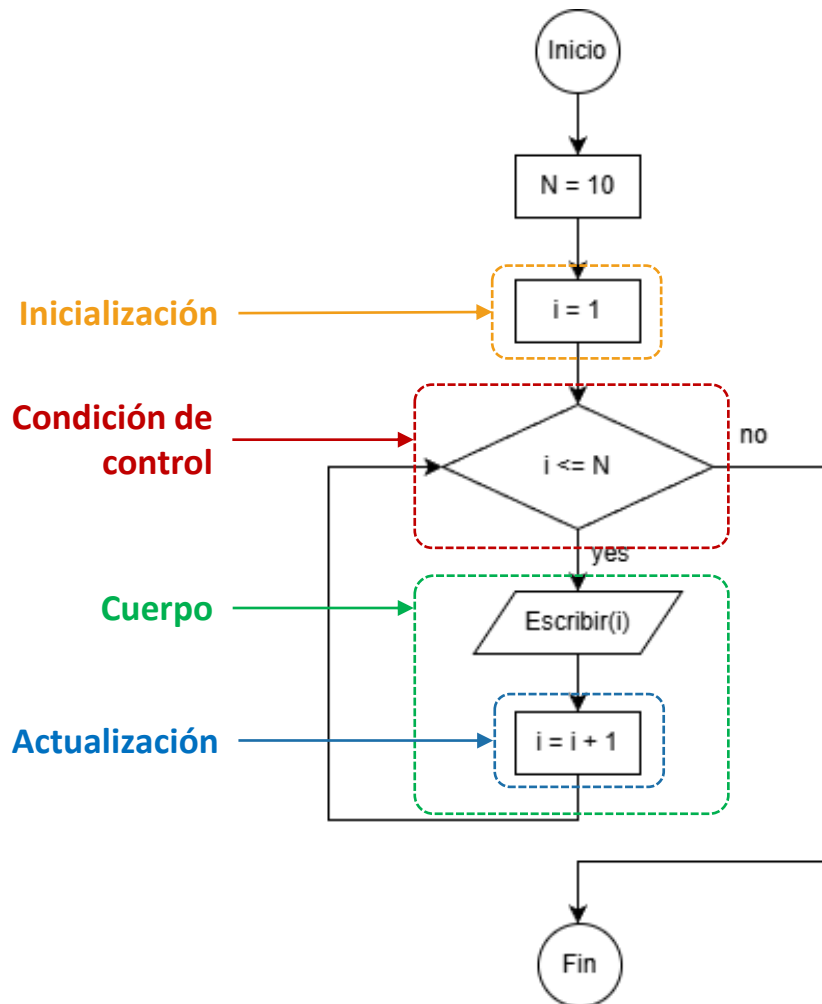
```
for vc in range(inicio, fin + 1, actualizacion):  
    instrucciones cuerpo
```

Observaciones importantes:

- A diferencia del ciclo **Mientras**, el ciclo **Para** se caracteriza por que la inicialización, la condición de control (o parada) y la actualización de la variable de control (**VC**) se definen en la cabecera del ciclo.
- Lo anterior hace que la actualización sea un proceso automático, lo cual lo hace la opción mas apropiada para ciclos donde el numero de iteraciones es conocido.
- En python la implementación del **Para**, es diferente a la de lenguajes como C++ y Java, por lo que hay que tener en cuenta unos detalles adicionales que veremos luego.

Ciclo Para - Ejemplo 1

Retomando el problema inicial de imprimir los $N = 10$ primeros números, el uso de ciclos permite solucionar fácilmente la cantidad de números a imprimir cambiando solo el valor de N .



Ciclo Para – Ejemplo 1

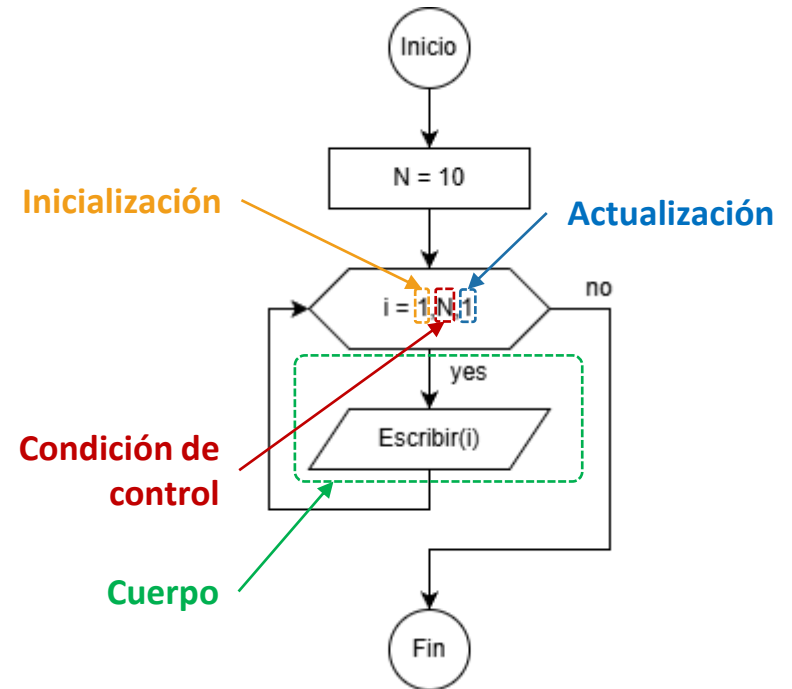
Pseudocódigo

```
Inicio
  N = 10
  Para (i = 1,N,1) Haga
    Escribir(i)
  Fin_Para
Fin
```

Python

```
N = 10
for i in range(1,N + 1,1):
    print(i)
```

[Link de ejecución](#)



Ciclos – Ejemplo 2

Escriba un algoritmo que calcule la suma de los primeros **N** números enteros mayores que 0, donde **N** es un dato de entrada.

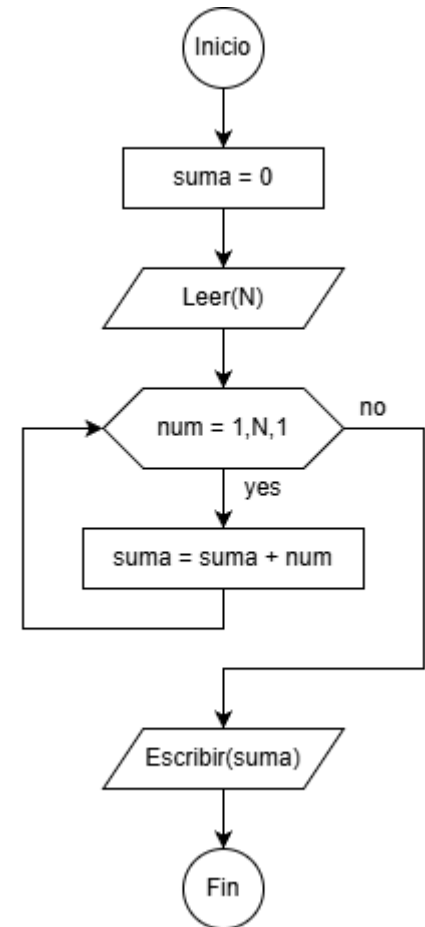
Pseudocódigo

```
Inicio
    suma = 0
    Leer(N)
    Para (num = 1,N,1) Haga
        suma = suma + num
    Fin_Para
    Escribir(suma)
Fin
```

Python

```
suma = 0
N = int(input('Ingrese la cantidad de números a sumar: '))
for num in range(1,N + 1):
    suma = suma + num
print(f"La suma de los {N} primeros números da {suma}")
```

[Link de ejecución](#)



Ciclos – Ejemplo 2

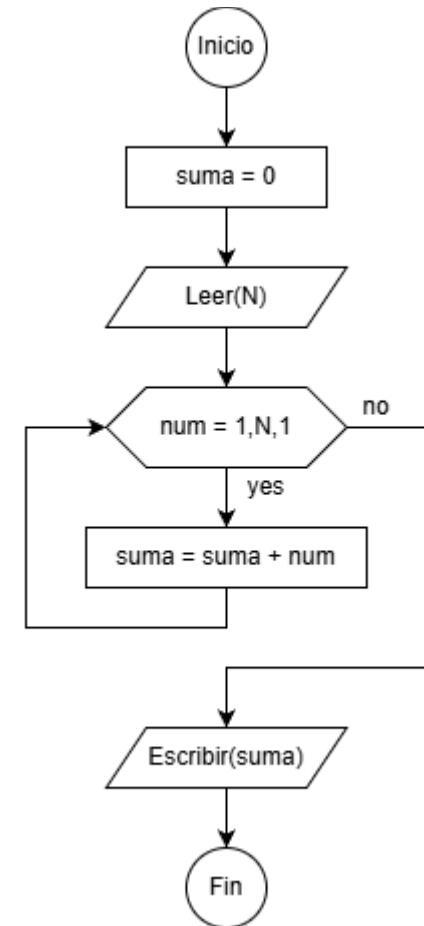
Escriba un algoritmo que calcule la suma de los primeros **N** números enteros mayores que 0, donde **N** es un dato de entrada.

```
Inicio
  suma = 0
  Leer(N)
  Para (num = 1, N + 1, 1) Haga
    suma = suma + num
  Fin_Para
  Escribir(suma)
Fin
```

Prueba de escritorio

N	suma
0	0
4	10

N	num	suma



Ciclos – Ejemplo 2

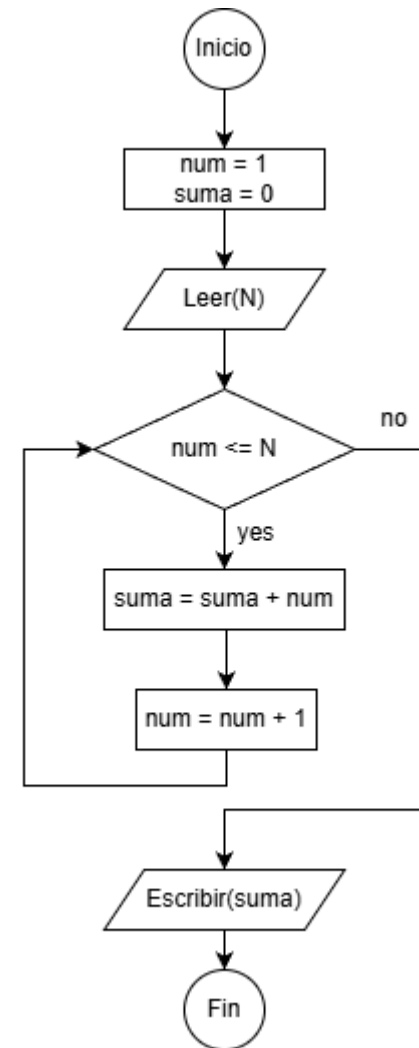
Escriba un algoritmo que calcule la suma de los primeros **N** números enteros mayores que 0, donde **N** es un dato de entrada.

```
Inicio
  suma = 0
  Leer(N)
  Para (num = 1,N,1) Haga
    suma = suma + num
  Fin_Para
  Escribir(suma)
Fin
```

Prueba de escritorio (solución)

N	suma
4	10

N	num	suma
?	?	?
4	1	0
	2	1
	3	3
	4	6
		10



Ciclos - Ejemplo 3

Haga un programa que calcule el factorial de un número entero no negativo n .

Recuerde cuando $n = 0$, $0! = 1$, y para cualquier otro valor el factorial esta dado por:

$$n! = 1 \times 2 \times 3 \times \cdots \times (n - 2) \times (n - 1) \times n$$

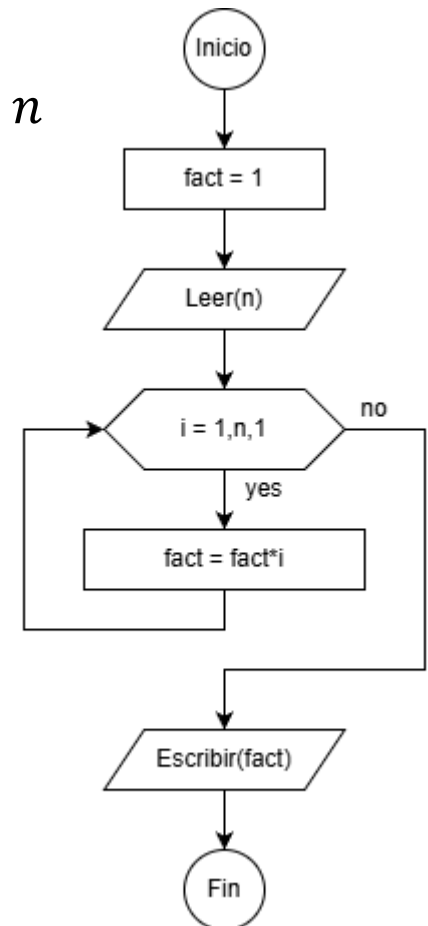
Pseudocódigo

```
Inicio
  fact = 1
  Leer(n)
  Para (i = 1,n,1) Haga
    fact = fact * i
  Fin_Para
  Escribir(n, '! = ',fact)
Fin
```

Python

```
fact = 1
n = int(input('Digite el numero (>= 0): '))
for i in range(1,n + 1):
    fact = fact * i
print(f"{n}! = {fact}")
```

[Link de ejecución](#)



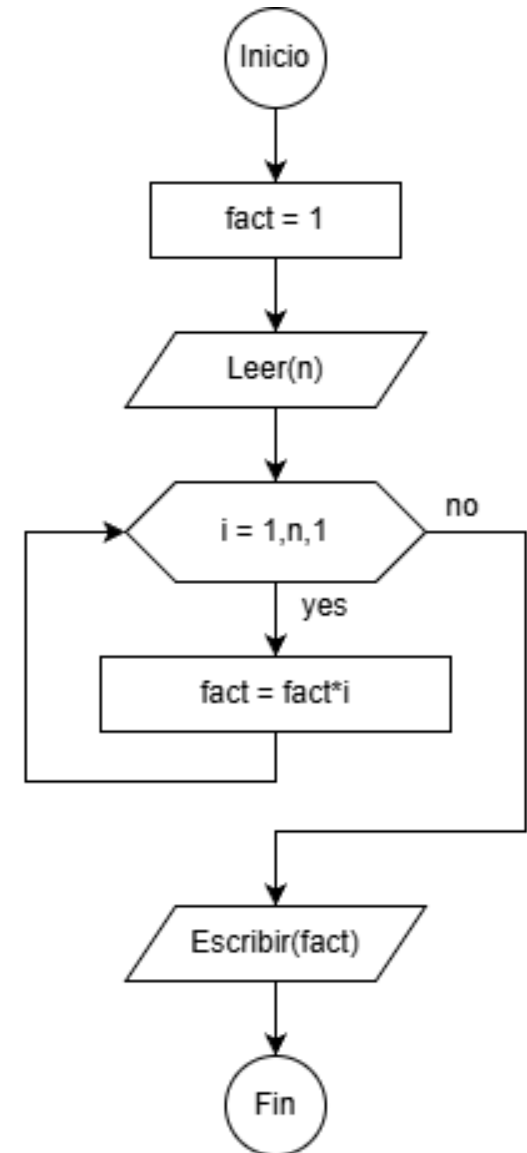
Ciclos – Ejemplo 3

```
Inicio
  fact = 1
  Leer(n)
  Para (i = 1,n,1) Haga
    fact = fact * i
  Fin_Para
  Escribir(n, '! = ',fact)
Fin
```

Prueba de escritorio

n	factorial
0	1
3	6
5	120

n	i	fact



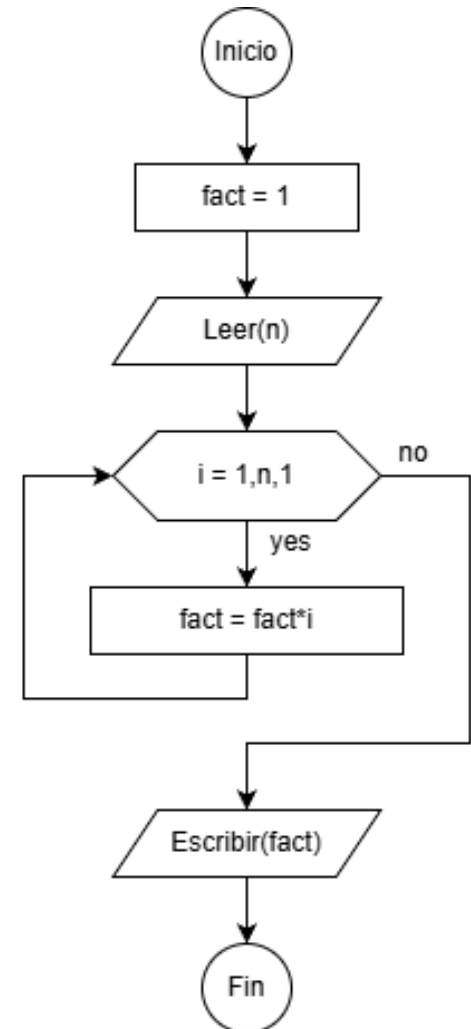
Ciclos – Ejemplo 3

```
Inicio
  fact = 1
  Leer(n)
  Para (i = 1,n,1) Haga
    fact = fact * i
  Fin_Para
  Escribir(n, '! = ',fact)
Fin
```

Prueba de escritorio (solución)

n	factorial
5	120

N	i	fact
?	?	?
5	1	1
	2	2
	3	6
	4	24
	5	120



Ciclos - Ejemplo 4

Imprimir los números del n a 1 alternando el signo, por ejemplo: 6, -5, 4, -3, 2, -1.

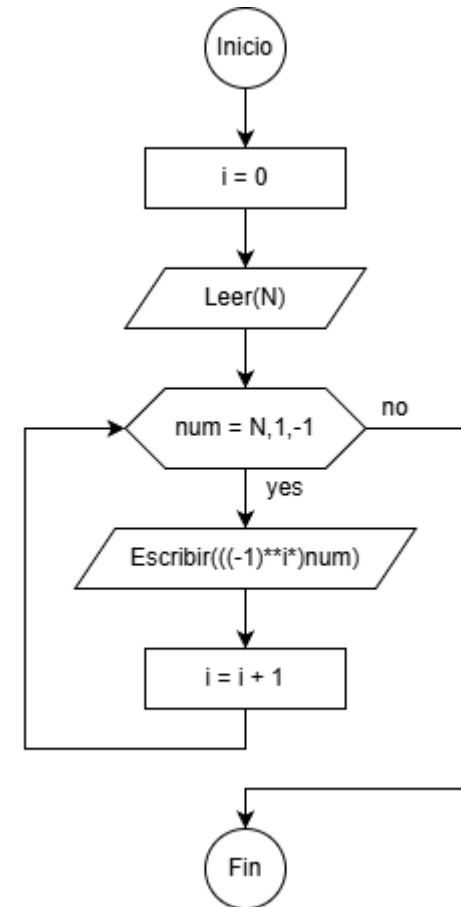
Pseudocódigo

```
Inicio
  i = 0
  Leer(N)
  Para (num = N, 1, -1) Haga
    Escribir(((−1)**i)*num)
    i = i + 1
  Fin_Para
Fin
```

Python

```
i = 0
N = int(input('Digite un numero entero positivo: '))
for num in range(N, 0, -1):
    print(((−1)**i)*num)
    i += 1
```

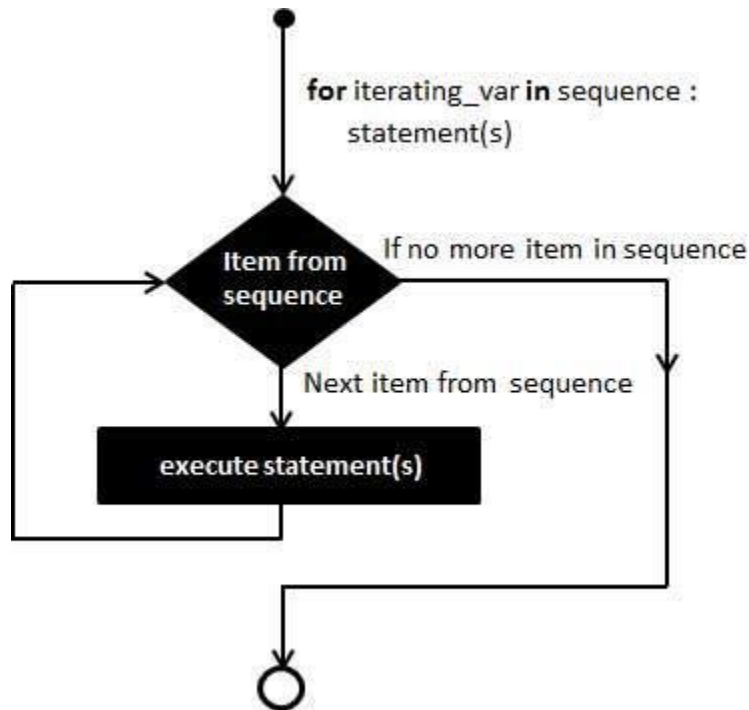
[Link de ejecución](#)



Nota: con actualización negativa, el límite de parada es **fin - 1** (no **fin + 1**), porque **range()** excluye ese valor en la dirección del paso. Por eso **range(N, 0, -1)** y no **range(N, 2, -1)**.

Sobre el ciclo para en python

- Como se dijo previamente, el ciclo Para en Python es diferente de la implementación en otros lenguajes como C, ya que la variable de control (**iterating_var**) itera sobre los elementos de una secuencia (**sequence**).



```
for iterating_var in sequence:
    statement(s)
```

¿Qué es una secuencia?

- Una secuencia es una colección ordenada de elementos, donde cada uno ocupa una posición (índice) que empieza en 0.
- El for de Python recorre una secuencia elemento por elemento (no cuenta posiciones ni necesita saber cuántas hay de antemano).
- Python tiene varios tipos de secuencia: cadenas, listas, tuplas, range, y otras para datos binarios (bytes, bytearray).
- En esta clase nos vamos a centrar en las tres que más se van usar por ahora: cadenas, listas y range.

```
# Cadena (str)
Lenguaje = "Python"

# List (list)
lenguajes = ["Python", "C", "C++", "Java"]

# range (range)
Numeros = range(1,5,1)
```

0	1	2	3	4	5
P	y	t	h	o	n
-6	-5	-4	-3	-2	-1

0	1	2	3
'Python'	'C'	'C++'	'Java'
-4	-3	-2	-1

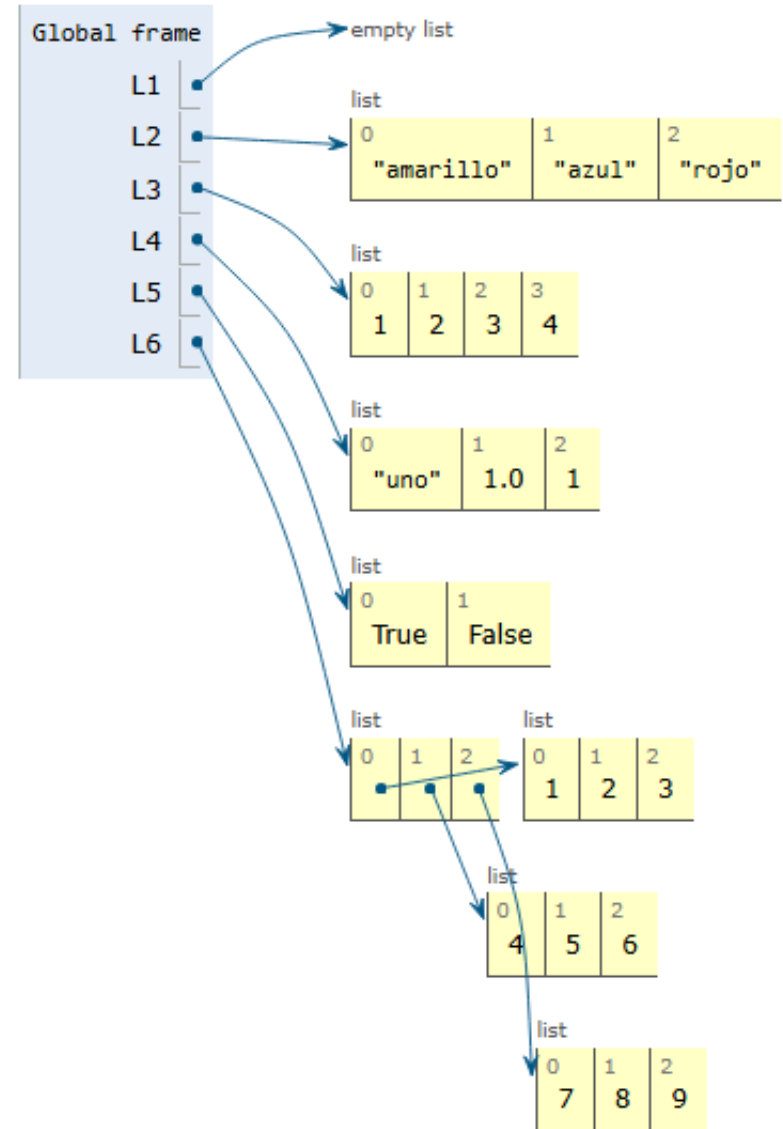
0	1	2	3
1	2	3	4
-4	-3	-2	-1

Secuencias: Listas

- Una lista es una secuencia **mutable** (que se puede cambiar) y ordenada de elementos, escrita entre corchetes [].
- Puede contener cualquier tipo de dato (enteros, flotantes, cadenas o incluso diferentes tipos).
- La lista es por así decirlo, el equivalente en programación (sin ser estricto) a los arreglos.
- Una lista puede almacenar otras listas.

```
L1 = []  
L2 = ["amarillo", "azul", "rojo"]  
L3 = [1, 2, 3, 4]  
L4 = ["uno", 1.0, 1]  
L5 = [True, False]  
L6 = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

[Link de ejecución](#)



Secuencias: Cadenas

- Como vimos al principio del curso, una cadena es una secuencia inmutable (no se puede cambiar) y ordenada de caracteres, escrita entre comillas simples ' ' o dobles " ".
- Se comporta como una lista de caracteres: se puede recorrer, indexar y medir con `len()`, pero a diferencia de la lista, no se puede modificar directamente.
- Aunque "contenga" números, una cadena sigue siendo texto: "**123**" no es lo mismo que el entero **123**.
- Con comillas triples (' ' ' o " " ") se pueden escribir cadenas de varias líneas.

```
C1 = ""  
C2 = "Hola mundo"  
C3 = 'Python'  
C4 = "123"  
C5 = "Línea 1\nLínea 2"  
C6 = """Varias  
líneas"""
```

[Link de ejecución](#)

Frames

Objects

Global frame

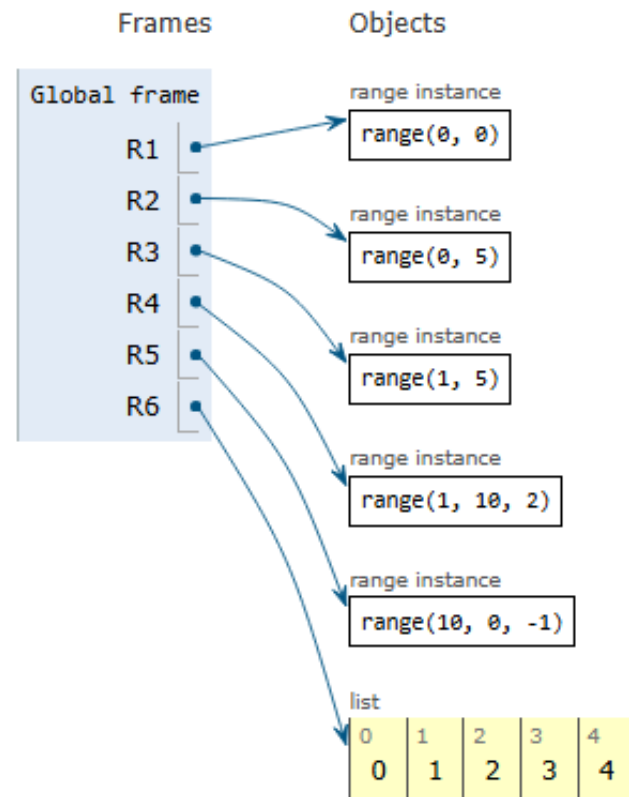
C1	""
C2	"Hola mundo"
C3	"Python"
C4	"123"
C5	"Línea 1 Línea 2"
C6	"Varias líneas"

Secuencias: range

- `range()` genera una secuencia de números enteros; no acepta decimales.
- **No es una lista:** es un objeto de tipo range (inmutable). Para ver sus valores como lista, se usa `list(...)`.
- **Puede recibir uno, dos o tres argumentos:** `range(fin)`, `range(inicio, fin)` o `range(inicio, fin, paso)`. El límite final siempre queda excluido.
- El paso puede ser negativo, para generar una secuencia descendente.

```
R1 = range(0)
R2 = range(5)
R3 = range(1, 5)
R4 = range(1, 10, 2)
R5 = range(10, 0, -1)
R6 = list(range(5))
```

[Link de ejecución](#)



El mismo for para tres secuencias distintas

- Sin importar el tipo de secuencia, el **for** de Python usa siempre la misma sintaxis. Lo único que cambia es la secuencia.

```
for iterating_var in sequence:  
    statemen(s)
```

Lista

```
colores = ["Amarillo", "Azul", "Rojo"]  
for color in colores:  
    print(color)
```

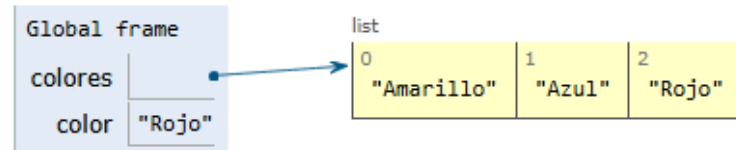
[Link de ejecución](#)

Print output (drag lower right corner to resize)

```
Amarillo  
Azul  
Rojo
```

Frames

Objects



Cadena

```
saludo = "Hola mundo"  
for letra in saludo:  
    print(letra, end = " ")
```

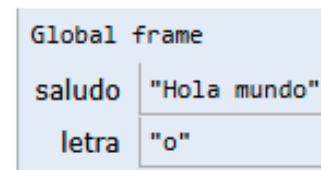
[Link de ejecución](#)

Print output (drag lower right corner to resize)

```
H o l a   m u n d o
```

Frames

Objects



El mismo for para tres secuencias distintas

- Sin importar el tipo de secuencia, el **for** de Python usa siempre la misma sintaxis. Lo único que cambia es la secuencia.

```
for iterating_var in sequence:  
    statement(s)
```

Rango

```
rango = range(1,11)  
for num in rango:  
    print(num)
```

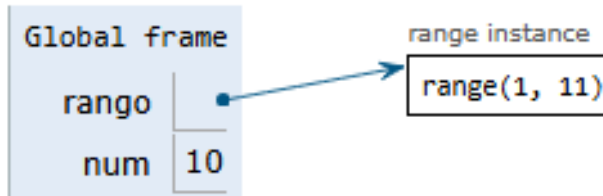
[Link de ejecución](#)

Print output (drag lower right corner to resize)

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

Frames

Objects



Ciclos – Ejemplo 5

Hacer un programa que lea primero el número de estudiantes de un curso (**N**), y luego lea las **N** notas correspondientes (valores decimales entre 0.0 y 5.0), una a una.

Considere que la nota mínima aprobatoria es 3.0: una nota es aprobada si es mayor o igual a 3.0, y reprobada si es menor a 3.0.

Al finalizar la lectura, el programa deberá entregar:

- El porcentaje de estudiantes aprobados.
- El porcentaje de estudiantes reprobados.
- El promedio de las notas aprobadas.
- El promedio de las notas reprobadas.
- El promedio de las notas del curso.

Ciclos – Ejemplo 5



Datos de entrada:

- Cantidad de estudiantes
- Nota del estudiante

Datos de salida:

- Porcentaje de estudiantes que ganaron
- Porcentaje de estudiantes que perdieron
- Promedio de notas de los que ganaron
- Promedio de notas de los que perdieron
- Promedio de notas del curso.

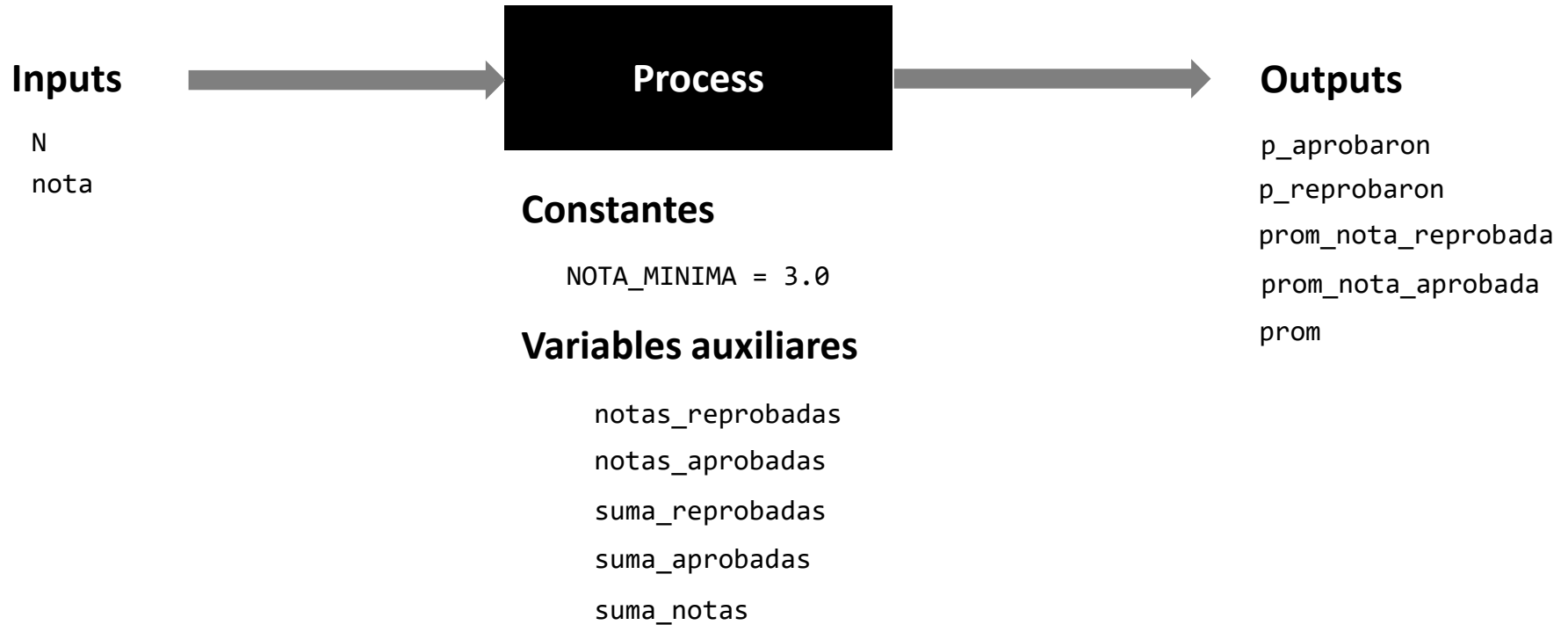
Definición de variables:

- **Entrada/salida:**
 - **N:** Numero de estudiantes
 - **nota:** Nota del estudiante
 - **p_aprobaron:** Porcentaje ganadores.
 - **p_reprobaron:** Porcentaje perdedores.
 - **prom_nota_reprobada:** Promedio notas reprobadas.
 - **prom_nota_aprobada:** Promedio de notas aprobadas.
 - **prom:** Promedio de notas del curso

Definición de variables:

- **Variables auxiliares:**
 - **i:** Contador asociado a la i-esima nota
 - **notas_reprobadas:** Numero de notas reprobadas (contador)
 - **suma_reprobadas:** Suma de notas reprobadas (acumulador).
 - **notas_aprobadas:** Numero de notas aprobadas (contador)
 - **suma_aprobadas:** Suma de notas reprobadas (acumulador).
 - **suma_notas:** Suma de las notas registradas.

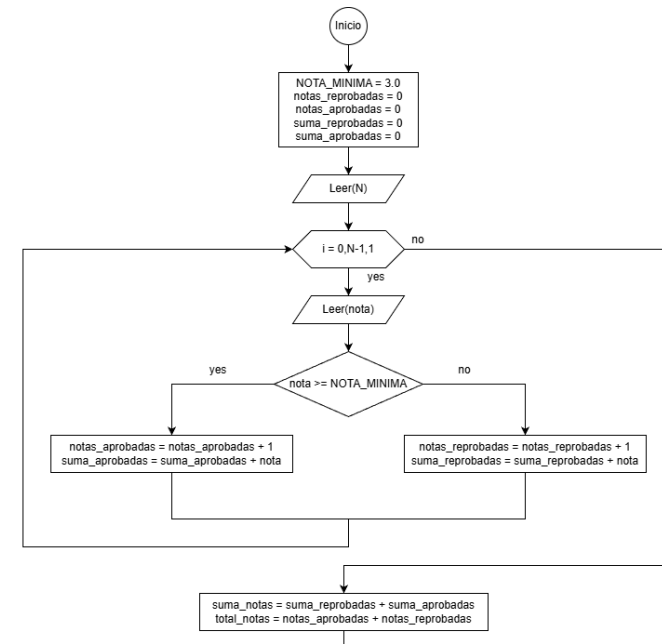
Ciclos – Ejemplo 5



Ciclos – Ejemplo 5

Implementación de la solución - parte 1:

- Inicialización.
- Para cada uno de los estudiantes:
 - Entrada de la nota de cada estudiante.
 - Actualización de contadores y acumuladores de acuerdo a la nota.



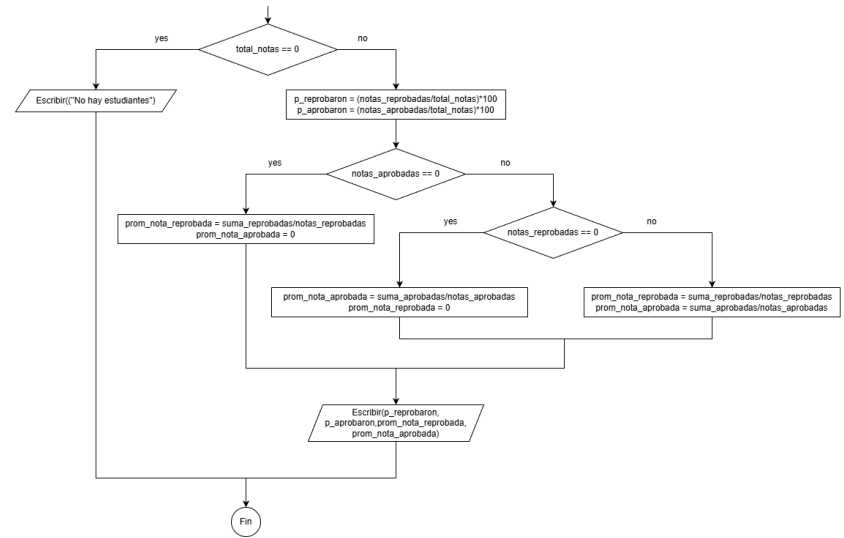
```
Inicio
NOTA_MINIMA = 3.0
notas_reprobadas = 0
notas_aprobadas = 0
suma_reprobadas = 0
suma_aprobadas = 0
Leer(N)
Para (i = 0, N - 1, 1) Haga
    Leer(nota)
    Si nota >= NOTA_MINIMA Entonces
        notas_aprobadas = notas_aprobadas + 1
        suma_aprobadas = suma_aprobadas + nota
    Sino
        notas_reprobadas = notas_reprobadas + 1
        suma_reprobadas = suma_reprobadas + nota
    Fin_Si
Fin_Para
suma_notas = suma_reprobadas + suma_aprobadas
total_notas = notas_aprobadas + notas_reprobadas
...
Fin
```

```
NOTA_MINIMA = 3.0
notas_reprobadas = 0
notas_aprobadas = 0
suma_reprobadas = 0
suma_aprobadas = 0
N = int(input('Ingrese el número de estudiantes: '))
for i in range(N):
    nota = float(input('Ingrese la nota: '))
    if nota >= NOTA_MINIMA:
        notas_aprobadas = notas_aprobadas + 1
        suma_aprobadas = suma_aprobadas + nota
    else:
        notas_reprobadas = notas_reprobadas + 1
        suma_reprobadas = suma_reprobadas + nota
    suma_notas = suma_reprobadas + suma_aprobadas
    total_notas = notas_aprobadas + notas_reprobadas
    ...
```

Ciclos – Ejemplo 5

Implementación de la solución - parte 2:

- Calculo de estadísticas definitivas.
- Despliegue de las estadísticas del grupo.



```
...
Si total_notas == 0 Entonces
    Escribir("No hay estudiantes")
Sino
    p_aprobaron = (notas_aprobadas/total_notas)*100
    p_reprobaron = (notas_reprobadas/total_notas)*100
    Si (notas_aprobadas == 0) Entonces
        prom_not_reprobada = suma_reprobadas/notas_reprobadas
        prom_not_aprobada = 0
    Sino
        Si (notas_reprobadas == 0) Entonces
            prom_not_aprobada = suma_aprobadas/notas_aprobadas
            prom_not_reprobada = 0
        Sino
            prom_not_reprobada = suma_reprobadas/notas_reprobadas
            prom_not_aprobada = suma_aprobadas/notas_aprobadas
        Fin_Si
    Fin_Si
    prom = suma_notas/total_notas
    Escribir(p_aprobaron, p_reprobaron)
    Escribir(prom_not_reprobada, prom_not_aprobada)
    Escribir(prom)
Fin_Si
Fin
```

```
...
if total_notas == 0:
    print("No hay estudiantes")
else:
    p_aprobaron = (notas_aprobadas/total_notas)*100
    p_reprobaron = (notas_reprobadas/total_notas)*100
    if (notas_aprobadas == 0):
        prom_not_reprobada = suma_reprobadas/notas_reprobadas
        prom_not_aprobada = 0
    elif (notas_reprobadas == 0):
        prom_not_aprobada = suma_aprobadas/notas_aprobadas
        prom_not_reprobada = 0
    else:
        prom_not_reprobada = suma_reprobadas/notas_reprobadas
        prom_not_aprobada = suma_aprobadas/notas_aprobadas
    prom = suma_notas/total_notas
    print(f"Porcentaje aprobados: {p_aprobaron}%")
    print(f"Porcentaje reprobados: {p_reprobaron}%")
    print(f"Promedio reprobados: {prom_not_reprobada}")
    print(f"Promedio aprobadas: {prom_not_aprobada}")
    print(f"Promedio de notas: {prom}")
```

Ciclos – Ejemplo 5

Resultados esperados

Numero de estudiantes	5
Notas	4.0, 2.5, 3.0, 1.5, 4.8
Aprobaron	(3) 60%
Reprobaron	(2) 40%
Promedio nota aprobada	3.93
Promedio nota reprobada	2.0
Promedio general	3.16

Prueba de escritorio

Realizar la prueba de escritorio para verificar que el algoritmo este bien planteado

Nota: Este programa así como esta planeado tiene aun un bug grave. Encuéntrelo y corrija.

```
NOTA_MINIMA = 3.0
notas_reprobadas = 0
notas_aprobadas = 0
suma_reprobadas = 0
suma_aprobadas = 0
N = int(input('Ingrese el número de estudiantes: '))
for i in range(N):
    nota = float(input('Ingrese la nota: '))
    if nota >= NOTA_MINIMA:
        notas_aprobadas = notas_aprobadas + 1
        suma_aprobadas = suma_aprobadas + nota
    else:
        notas_reprobadas = notas_reprobadas + 1
        suma_reprobadas = suma_reprobadas + nota
suma_notas = suma_reprobadas + suma_aprobadas
total_notas = notas_aprobadas + notas_reprobadas
if total_notas == 0:
    print("No hay estudiantes")
else:
    p_aprobaron = (notas_aprobadas/total_notas)*100
    p_reprobaron = (notas_reprobadas/total_notas)*100
    if (notas_aprobadas == 0):
        prom_nota_reprobada = suma_reprobadas/notas_reprobadas
        prom_nota_aprobada = 0
    elif (notas_reprobadas == 0):
        prom_nota_aprobada = suma_aprobadas/notas_aprobadas
        prom_nota_reprobada = 0
    else:
        prom_nota_reprobada = suma_reprobadas/notas_reprobadas
        prom_nota_aprobada = suma_aprobadas/notas_aprobadas
    prom = suma_notas/total_notas
    print(f"Porcentaje aprobados: {p_aprobaron}%")
    print(f"Porcentaje reprobados: {p_reprobaron}%")
    print(f"Promedio reprobados: {prom_nota_reprobada}")
    print(f"Promedio aprobados: {prom_nota_aprobada}")
    print(f"Promedio de notas: {prom}")
```

Ciclos – Ejemplo 5

Resultados esperados

Numero de estudiantes	5
Notas	4.0, 2.5, 3.0, 1.5, 4.8
Aprobaron	(3) 60%
Reprobaron	(2) 40%
Promedio nota aprobada	3.93
Promedio nota reprobada	2.0
Promedio general	3.16

Forma

```
i = 0
...
Para (i = 0,N,1) Haga
    Leer(nota)
    ...
Fin_Para
...
```

Resultado de ejecución

```
Ingrese el número de estudiantes: 5
Ingrese la nota del estudiante 1: 4
Ingrese la nota del estudiante 2: 2.5
Ingrese la nota del estudiante 3: 3.0
Ingrese la nota del estudiante 4: 1.5
Ingrese la nota del estudiante 5: 4.8
Porcentaje de estudiantes que aprobaron: 60.0%
Porcentaje de estudiantes que reprobaron: 40.0%
Promedio de notas reprobadas: 2.0
Promedio de notas aprobadas: 3.9333333333333336
Promedio general de notas: 3.16
```

Ciclos – Ejemplo 6

Realizar un programa que muestre cada uno de los divisores de un numero positivo ingresado por teclado. Al final, el programa también deberá mostrar la cantidad total de divisores asociados al numero.



Datos de entrada:

- Numero

Datos de salida:

- Divisores del numero

Definición de variables:

- **num:** Numero ingresado por el usuario.
- **i:** Divisor i-esimo.
- **num_divisores:** Cantidad de divisores del numero.

Ciclos – Ejemplo 6

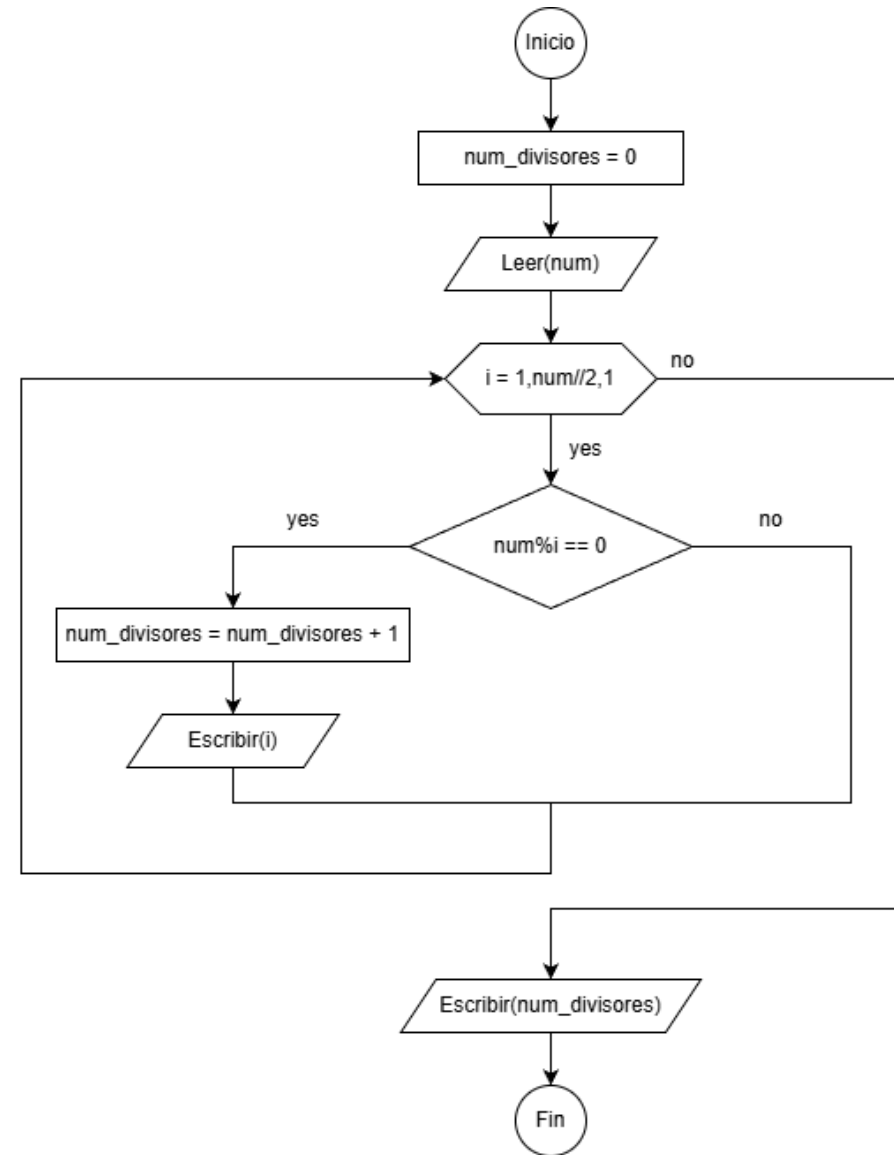
Pseudocódigo

```
Inicio
  num_divisores = 0
  Leer(num)
  Para (i = 1, num//2, 1) Haga
    Si (num%i == 0) Entonces
      num_divisores = num_divisores + 1
      Escribir(i)
    Fin_Si
  Fin_Para
  Escribir(num_divisores)
Fin
```

Python

```
i = 0
N = int(input('Digite un numero entero positivo: '))
for i in range(1, num//2 + 1):
    if num % i == 0:
        num_divisores += 1
        print(f"{i} es divisor de {num}")
print(f"{num} tiene {num_divisores} divisores")
```

[Link de ejecución](#)



Ciclos – Ejemplo 7

Realizar un programa que indique si un numero ingresado por el usuario es o no es primo.



Datos de entrada:

- **Numero**

Datos de salida:

- **Indicación si es o no es primo**

Definición de variables:

- **num:** Numero ingresado por el usuario.
- **i:** Divisor i-esimo.
- **num_divisores:** Cantidad de divisores del numero.

Ciclos – Ejemplo 7

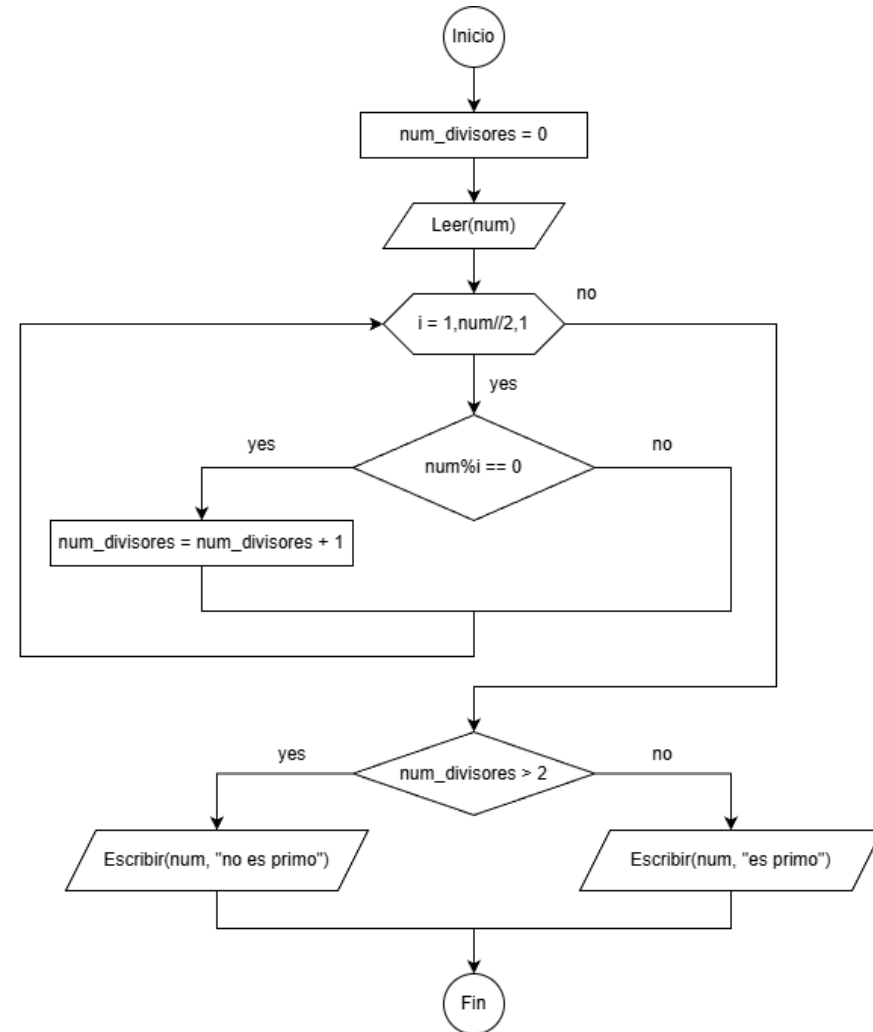
Pseudocódigo

```
Inicio
  num_divisores = 0
  Leer(num)
  Para (i = 1, num//2, 1) Haga
    Si (num%i == 0) Entonces
      num_divisores = num_divisores + 1
    Fin_Si
  Fin_Para
  Si(num_divisores > 2) Entonces
    Escribir(num, " no es primo")
  Sino
    Escribir(num, " es primo")
  Fin_Si
  Escribir(num_divisores)
Fin
```

Python

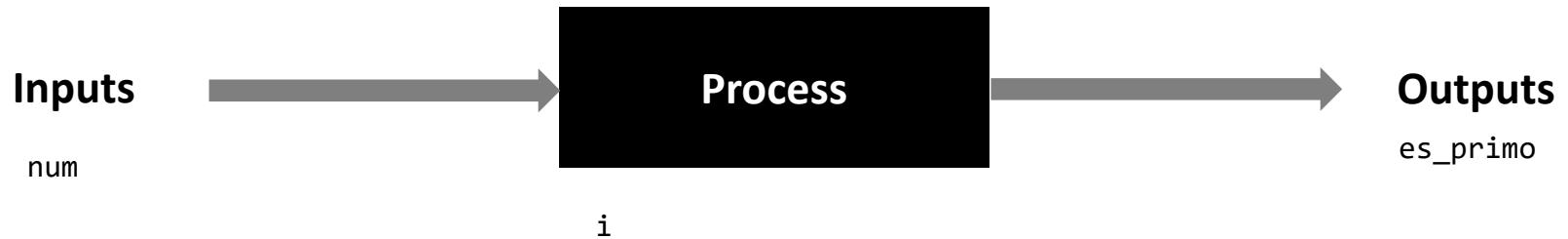
```
i = 0
N = int(input('Digite un numero entero positivo: '))
for i in range(1, num//2 + 1):
    if num % i == 0:
        num_divisores += 1
if (num_divisores > 2):
    print(num, " no es primo")
else:
    print(num, " es primo")
```

[Link de ejecución](#)



Ciclos – Ejemplo 7 (Otra forma)

Realizar un programa que indique si un numero ingresado por el usuario es o no es primo.



En este caso se uso una bandera (`es_primo`) y ruptura de ciclos para hacer mas eficiente el programa.

Datos de entrada:

- Numero

Datos de salida:

- Indicación si es o no es primo

Definición de variables:

- `num`: Numero ingresado por el usuario.
- `i`: Divisor-iesimo asociado al numero.
- `es_primo`: Bandera que indica si el numero es o no primo.

Ciclos – Ejemplo 7 (Otra forma)

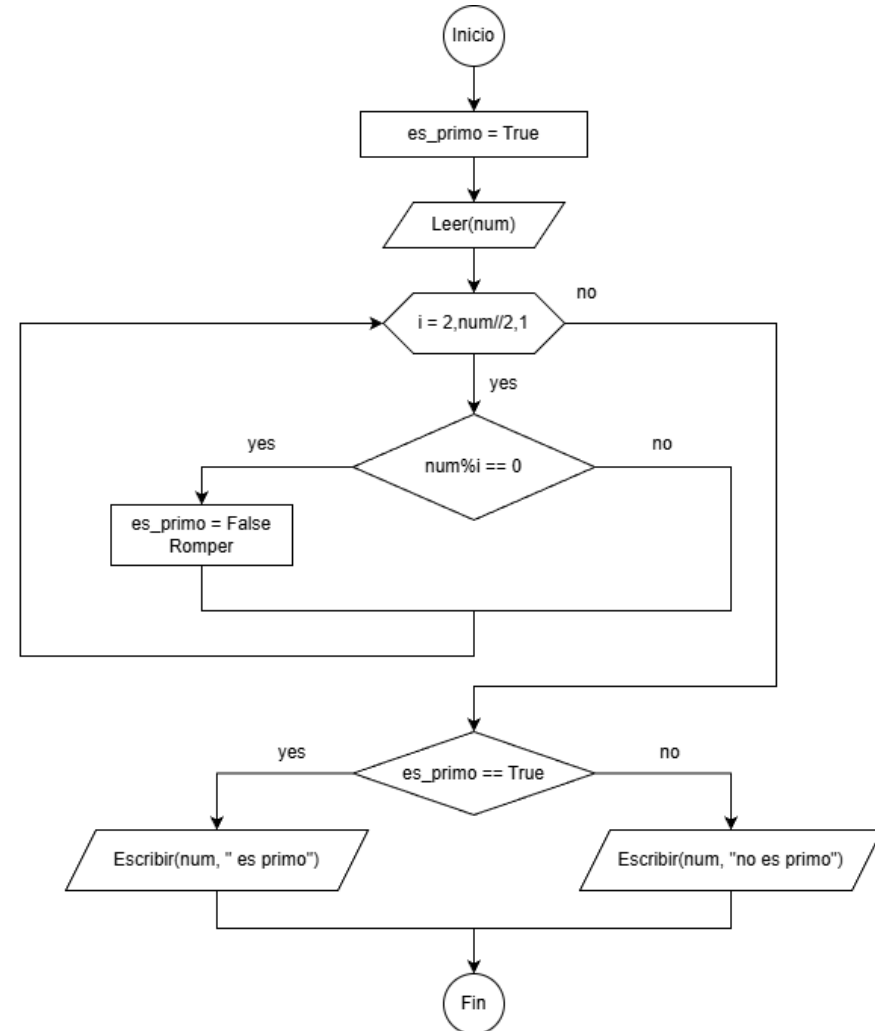
Pseudocódigo

```
Inicio
  es_primo = True
  Leer(num)
  Para (i = 2, num//2, 1) Haga
    Si (num%i == 0) Entonces
      es_primo = False
      Romper
    Fin_Si
  Fin_Para
  Si(es_primo == True) Entonces
    Escribir(num, " es primo")
  Sino
    Escribir(num, " no es primo")
  Fin_Si
  Escribir(num_divisores)
Fin
```

Python

```
es_primo = True
N = int(input('Digite un numero entero positivo: '))
for i in range(2, num//2 + 1):
    if num % i == 0:
        es_primo = False
        break
if (es_primo):
    print(num, " es primo")
else:
    print(num, " no es primo")
```

[Link de ejecución](#)



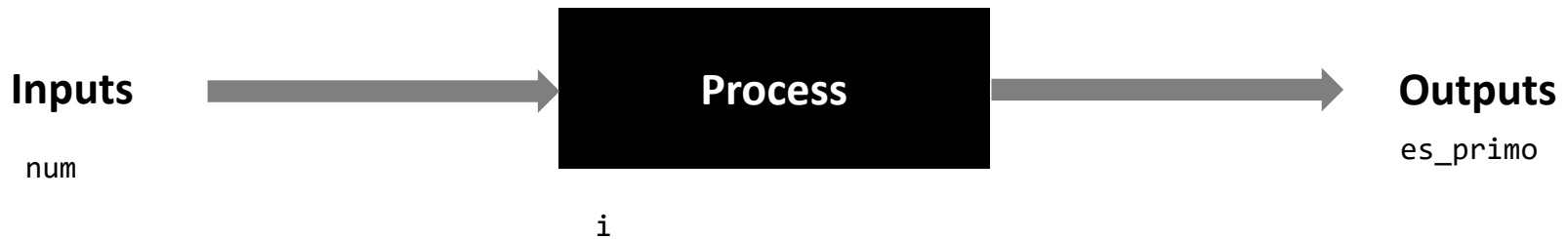
Ciclos – Ejemplo 8

La sucesión de Fibonacci es una secuencia de números en la que cada término se obtiene sumando los dos términos anteriores:

$$f_n = f_{n-1} + f_{n-2}$$

Con los valores iniciales $f_0 = 0$ y $f_1 = 1$.

Hacer un programa que reciba un número entero positivo **n** y muestre los primeros **n** términos de la sucesión de Fibonacci.



Datos de entrada:

- **Numero**

Datos de salida:

- **Indicación si es o no es primo**

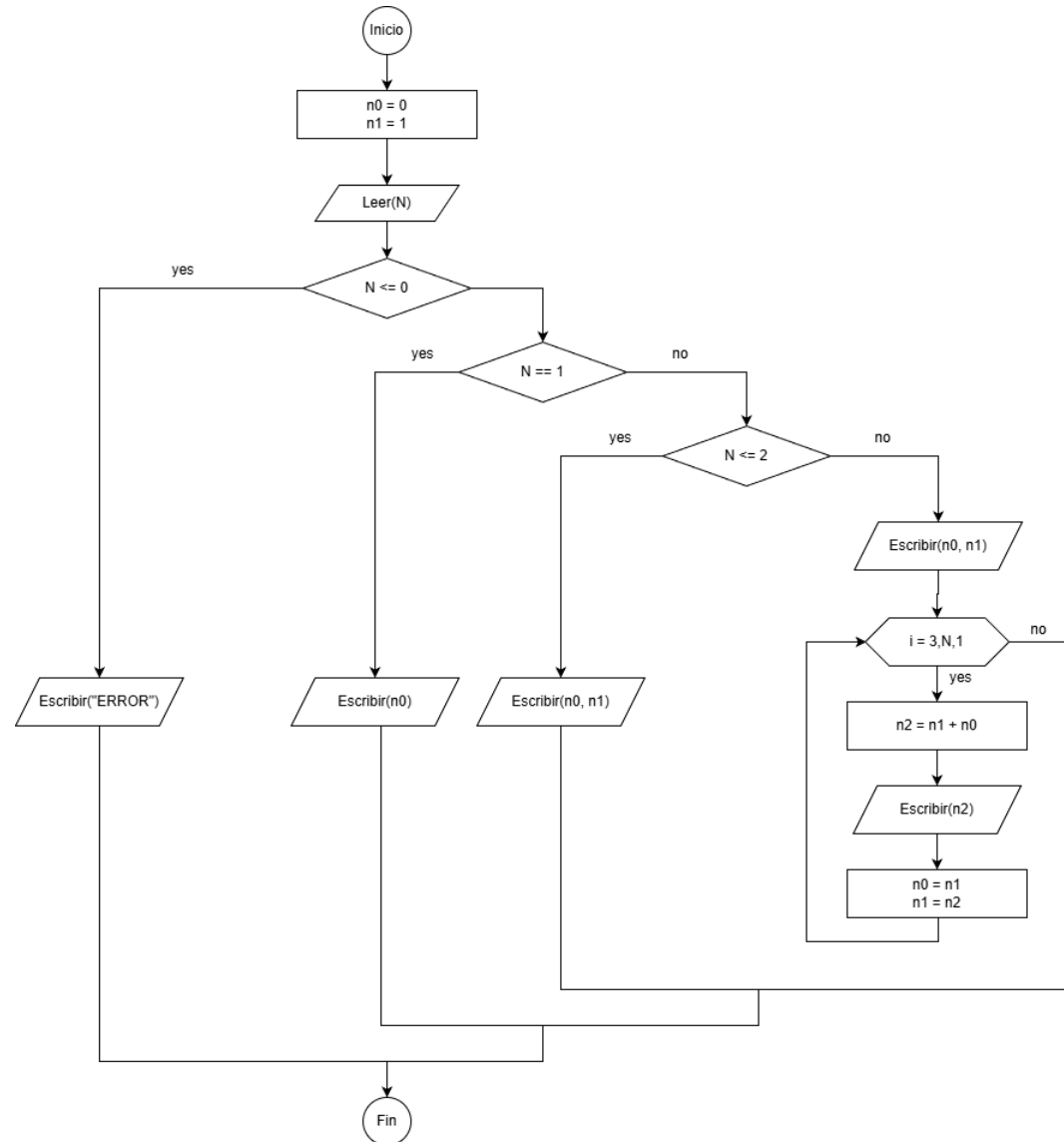
Definición de variables:

- **num**: Numero ingresado por el usuario.
- **i**: Divisor i-esimo
- **es_primo**: Bandera que indica si el numero es o no primo.

Ciclos – Ejemplo 8

Pseudocódigo

```
Inicio
  n0 = 0
  n1 = 1
  Leer(N)
  Si N <= 0 Entonces
    Escribir("ERROR")
  Sino
    Si N == 1 Entonces
      Escribir(n0)
    Sino
      Si N <= 2 Entonces
        Escribir(n0, n1)
      Sino
        Escribir(n0, n1)
        Para (i = 3, N, 1) Haga
          n2 = n1 + n0
          Escribir(n2)
          n0 = n1
          n1 = n2
        Fin_Para
      Fin_Si
    Fin_Si
  Fin_Si
Fin
```

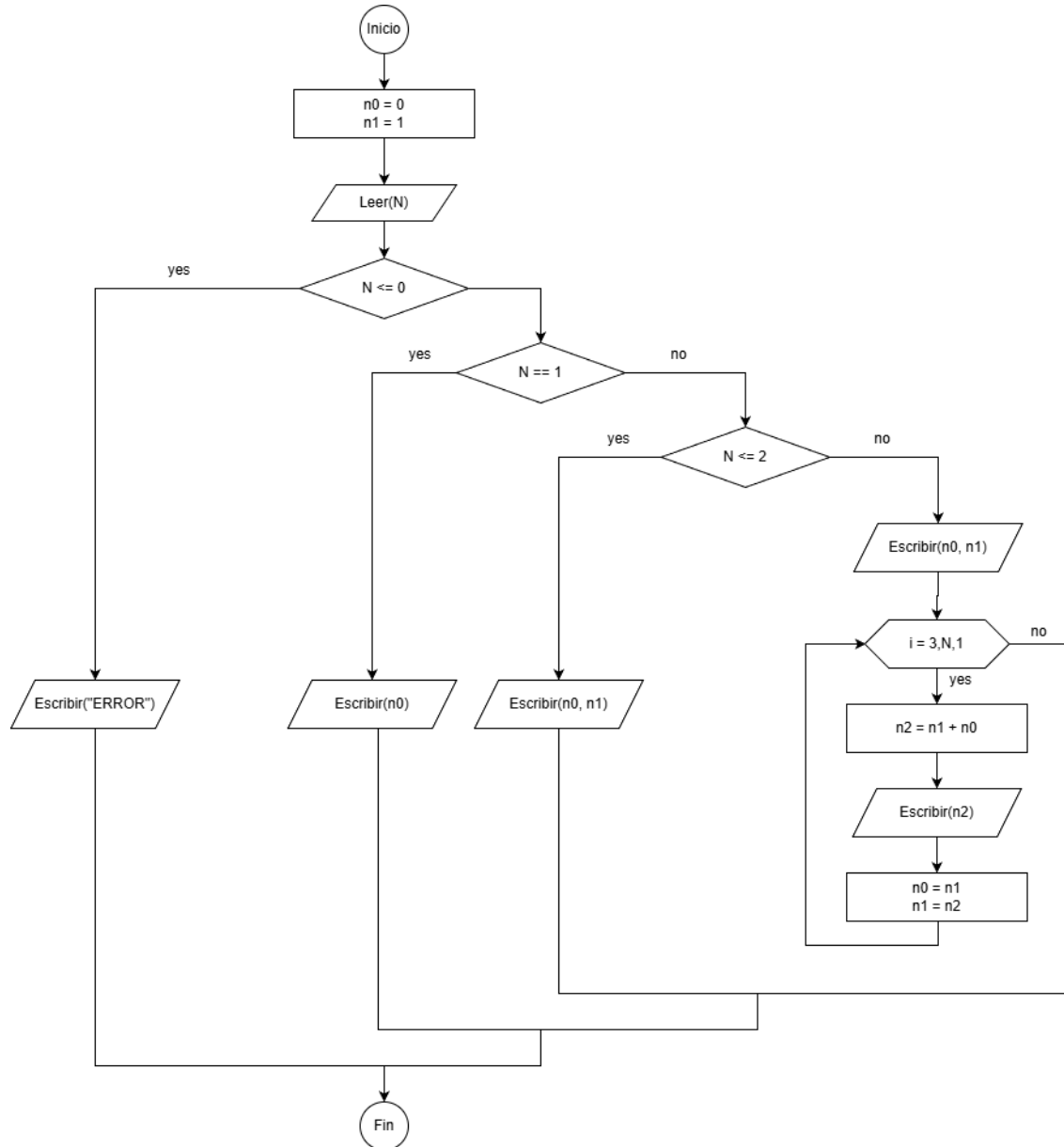


Ciclos – Ejemplo 8

Python

```
n0 = 0
n1 = 1
N = int(input('Ingrese N: '))
if N <= 0:
    print("ERROR")
elif N == 1:
    print(n0)
elif N <= 2:
    print(n0, n1)
else:
    print(n0, n1)
    for i in range(3, N + 1):
        n2 = n1 + n0
        print(n2)
        n0 = n1
        n1 = n2
```

[Link de ejecución](#)



Ciclos: algunos patrones analizados

- Hasta el momento, en lo que respecta al uso de estructuras repetitivas (ciclo) hemos visto dos tipos:
 - **Ciclo Mientras:** útil en problemas de iteración desconocida (terminación por bandera o centinela) y de iteración conocida (empleo de contadores).
 - **Ciclo Para:** útil para ciclos con un número conocido de iteraciones.
- Sobre esta base construimos una serie de ejemplos que fueron mostrando distintos patrones de uso de un ciclo (en ambos tipos), no solo su sintaxis. Más exactamente, en esta clase usamos el ciclo Para para resolver problemas de iteración conocida:
- El Ejemplo 2 introdujo el acumulador de suma.
- El Ejemplo 5 (notas de estudiantes) introdujo el acumulador condicional: contar y sumar por separado según se cumpla o no una condición (aprobada/reprobada).
- Los Ejemplos 6 y 7 (divisores, primalidad) trabajaron con conteo condicional; la variante "Ejemplo 7 (Otra forma)" combinó una bandera con ruptura del ciclo (break) para hacerlo más eficiente.
- Fibonacci introdujo el manejo de dos variables de estado simultáneas y la variable temporal.

Ciclos – Ejemplo 9

Hacer un programa que lea primero el número de estudiantes de un curso (**N**), y luego lea las **N** notas correspondientes (valores decimales entre 0.0 y 5.0), una a una.

Considere que la nota mínima aprobatoria es 3.0: una nota es aprobada si es mayor o igual a 3.0, y reprobada si es menor a 3.0.

Al finalizar la lectura, el programa deberá entregar:

- El porcentaje de estudiantes aprobados.
- El porcentaje de estudiantes reprobados.
- El promedio de las notas aprobadas.
- El promedio de las notas reprobadas.
- El promedio de las notas del curso.
- La máxima nota obtenida en el curso.
- La mínima nota obtenida en el curso.

Nota: Esta es una actualización que se realiza sobre el ejemplo 5.

Ciclos: Búsqueda de extremos

- **Idea central:** mantener una variable que representa el "**mejor**" valor visto hasta el momento (el máximo, o el mínimo), y en cada iteración compararla contra el nuevo dato.
- Si el nuevo dato "mejora" lo que ya se tiene guardado (es mayor, para el máximo; es menor, para el mínimo), se actualiza esa variable. Si no, se deja igual y se continúa con el siguiente dato.
- A diferencia de los acumuladores, aquí no se combina la información de todos los datos sino que solo se retiene uno: el mejor visto hasta ahora.
- En pseudocódigo, la esencia del patrón es esta (independiente de cómo se recorra la secuencia):

```
Mientras (i < N) Haga
  Leer(elemento)
  Si elemento > maximo Entonces
    maximo = elemento
  Fin_Si
  Si elemento < minimo Entonces
    minimo = elemento
  Fin_Si
  i = i + 1
Fin_Mientras
```

```
Para (i = 1,N,1) Haga
  Leer(elemento)
  Si elemento > maximo Entonces
    maximo = elemento
  Fin_Si
  Si elemento < minimo Entonces
    minimo = elemento
  Fin_Si
Fin_Para
```

Ciclos: Búsqueda de extremos (continuación)

- Pregunta clave: ¿con qué valor se inicializan máximo y mínimo antes de la primera comparación?
- Dos posibles respuestas:
 - **Opción 1 - valores fijos:** aprovechar que el dominio de las notas es conocido (0.0 a 5.0) e inicializar $\text{maximo} = 0.0$ y $\text{minimo} = 5.0$ antes del ciclo.
 - **Opción 2 - el primer dato leído:** inicializar maximo y minimo con el primer elemento de la secuencia, sin asumir nada sobre el rango de los datos.
- Analicemos cada una.

Opción 1: inicialización fija

- Se aprovecha que el dominio de los datos es conocido de antemano (en este caso, notas entre 0.0 y 5.0).
- Antes de iniciar el ciclo, **maximo** se inicializa con el valor más bajo posible del dominio, y **minimo** con el más alto — así, el primer dato real siempre "gana" la primera comparación.
- **Es la opción más simple:** no hay que tratar ninguna iteración de forma especial.
- **Limitación:** solo funciona si se conoce con certeza el rango válido de los datos.

```
i = 0
maximo = EXTREMO_INFERIOR
minimo = EXTREMO_SUPERIOR

Mientras (i < N) Haga
  Leer(elemento)
  Si elemento > maximo Entonces
    maximo = elemento
  Fin_Si
  Si elemento < minimo Entonces
    minimo = elemento
  Fin_Si
  i = i + 1
Fin_Mientras
```

```
maximo = EXTREMO_INFERIOR
minimo = EXTREMO_SUPERIOR

Para (i = 0, N - 1, 1) Haga
  Leer(elemento)
  Si elemento > maximo Entonces
    maximo = elemento
  Fin_Si
  Si elemento < minimo Entonces
    minimo = elemento
  Fin_Si
Fin_Para
```

Busqueda de extremos – Ejemplo 9 (Opción 1)

```
Inicio
    NOTA_MINIMA = 3.0
    MINIMO_VALIDO = 0.0
    MAXIMO_VALIDO = 5.0
    notas_reprobadas = 0
    notas_aprobadas = 0
    suma_reprobadas = 0
    suma_aprobadas = 0
    nota_maxima = MINIMO_VALIDO
    nota_minima = MAXIMO_VALIDO
    Leer(N)
    Para (i = 0, N - 1, 1) Haga
        Leer(nota)
        Si nota > nota_máxima Entonces
            nota_maxima = nota
        Fin_Si
        Si nota < nota_mínima Entonces
            nota_minimo = nota
        Fin_Si
        Si nota >= NOTA_MINIMA Entonces
            notas_aprobadas = notas_aprobadas + 1
            suma_aprobadas = suma_aprobadas + nota
        Sino
            notas_reprobadas = notas_reprobadas + 1
            suma_reprobadas = suma_reprobadas + nota
        Fin_Si
    Fin_Para
    suma_notas = suma_reprobadas + suma_aprobadas
    total_notas = notas_aprobadas + notas_reprobadas
    ...
Fin
```

```
NOTA_MINIMA = 3.0
MINIMO_VALIDO = 0.0
MAXIMO_VALIDO = 5.0
notas_reprobadas = 0
notas_aprobadas = 0
suma_reprobadas = 0
suma_aprobadas = 0
nota_maxima = MINIMO_VALIDO
nota_minima = MAXIMO_VALIDO

N = int(input('Ingrese el número de estudiantes: '))
for i in range(N):
    nota = float(input('Ingrese la nota: '))
    if nota > nota_máxima:
        nota_maxima = nota
    if nota < nota_mínima:
        nota_minimo = nota
    if nota >= NOTA_MINIMA:
        notas_aprobadas = notas_aprobadas + 1
        suma_aprobadas = suma_aprobadas + nota
    else:
        notas_reprobadas = notas_reprobadas + 1
        suma_reprobadas = suma_reprobadas + nota

suma_notas = suma_reprobadas + suma_aprobadas
total_notas = notas_aprobadas + notas_reprobadas
...
```

Opción 2: inicialización con el primer dato

- **No asume nada sobre el rango de los datos:** usa el primer elemento de la secuencia como punto de partida para máximo y mínimo.
- Requiere distinguir la primera iteración de las demás: en la primera, se asignan máximo y mínimo directamente; en las siguientes, se compara y se actualiza si corresponde.
- **Es más general:** se puede aplicar a cualquier problema, sin importar el dominio de los datos.

```
i = 0
Mientras (i < N) Haga
  Leer(elemento)
  Si i == 0 Entonces
    maximo = elemento
    minimo = elemento
  Sino
    Si elemento > maximo Entonces
      maximo = elemento
    Fin_Si
    Si elemento < minimo Entonces
      minimo = elemento
    Fin_Si
  Fin_Si
  i = i + 1
Fin_Mientras
```

```
Para (i = 0, N - 1, 1) Haga
  Leer(elemento)
  Si i == 0 Entonces
    maximo = elemento
    minimo = elemento
  Sino
    Si elemento > maximo Entonces
      maximo = elemento
    Fin_Si
    Si elemento < minimo Entonces
      minimo = elemento
    Fin_Si
  Fin_Si
Fin_Para
```

Busqueda de extremos – Ejemplo 9 (Opción 2)

```
Inicio
    NOTA_MINIMA = 3.0
    notas_reprobadas = 0
    notas_aprobadas = 0
    suma_reprobadas = 0
    suma_aprobadas = 0
    Leer(N)
    Para (i = 0, N - 1, 1) Haga
        Leer(nota)
        Si i == 0 Entonces
            nota_maxima = nota
            nota_minima = nota
        Sino
            Si nota > nota_maxima Entonces
                nota_maxima = nota
            Fin_Si
            Si nota < nota_minima Entonces
                nota_minima = nota
            Fin_Si
        Fin_Si
        Si nota >= NOTA_MINIMA Entonces
            notas_aprobadas = notas_aprobadas + 1
            suma_aprobadas = suma_aprobadas + nota
        Sino
            notas_reprobadas = notas_reprobadas + 1
            suma_reprobadas = suma_reprobadas + nota
        Fin_Si
    Fin_Para
    suma_notas = suma_reprobadas + suma_aprobadas
    total_notas = notas_aprobadas + notas_reprobadas
    ...
Fin
```

```
NOTA_MINIMA = 3.0
notas_reprobadas = 0
notas_aprobadas = 0
suma_reprobadas = 0
suma_aprobadas = 0

N = int(input('Ingrese el número de estudiantes: '))
for i in range(N):
    nota = float(input('Ingrese la nota: '))
    if i == 0:
        nota_maxima = nota
        nota_minima = nota
    else:
        if nota > nota_maxima:
            nota_maxima = nota
        if nota < nota_minima:
            nota_minima = nota
    if nota >= NOTA_MINIMA:
        notas_aprobadas = notas_aprobadas + 1
        suma_aprobadas = suma_aprobadas + nota
    else:
        notas_reprobadas = notas_reprobadas + 1
        suma_reprobadas = suma_reprobadas + nota

suma_notas = suma_reprobadas + suma_aprobadas
total_notas = notas_aprobadas + notas_reprobadas
...
```

Contextualización

- En el Ejemplo anterior se asumió que todas las notas ingresadas son válidas (están dentro del rango permitido).
- ¿Qué pasa si el usuario digita una nota fuera de rango, por ejemplo -1.0 o 6.0?
- Recordemos que en la clase anterior vimos cómo terminar un ciclo por completo antes de que se cumpla su condición natural, usando **Romper** (**break**).
- Ahora bien, no siempre queremos salir del ciclo por completo: a veces solo queremos ignorar el dato actual y seguir con el siguiente, sin detener todo el proceso.
- Para esto existe una instrucción que salta el resto del cuerpo del ciclo en la iteración actual, sin salir de él: **continue**.
- **Diferencia clave:**
 - **Romper:** **break** termina el ciclo por completo
 - **Continuar:** **continue** solo termina la iteración actual y sigue con la siguiente.

Caso break

```
i = 0
NUM_FINAL = 10
print("-----")
while i < NUM_FINAL:
    i += 1
    if (i >= 4) and (i <= 7):
        break
    print("i = " + str(i))
    print("-----")
```

[Link de ejecución](#)

Caso Continue

```
i = 0
NUM_FINAL = 10
print("-----")
while i < NUM_FINAL:
    i += 1
    if (i >= 4) and (i <= 7):
        continue
    print("i = " + str(i))
    print("-----")
```

[Link de ejecución](#)

Ejemplo 9: Aplicación de continue

Inicio

```
NOTA_MINIMA = 3.0
MINIMO_VALIDO = 0.0
MAXIMO_VALIDO = 5.0
notas_reprobadas = 0
notas_aprobadas = 0
suma_reprobadas = 0
suma_aprobadas = 0
notas_validas = 0
Leer(N)
Para (i = 0, N - 1, 1) Haga
    Leer(nota)
    Si nota < MINIMO_VALIDO or nota > MAXIMO_VALIDO Entonces
        Escribir("ERROR: nota fuera de rango")
        Continuar
    Fin_Si
    Si notas_validas == 0 Entonces
        nota_maxima = nota
        nota_minima = nota
    Sino
        Si nota > nota_maxima Entonces
            nota_maxima = nota
        Fin_Si
        Si nota < nota_minima Entonces
            nota_minima = nota
        Fin_Si
    Fin_Si
    notas_validas = notas_validas + 1
    Si nota >= NOTA_MINIMA Entonces
        notas_aprobadas = notas_aprobadas + 1
        suma_aprobadas = suma_aprobadas + nota
    Sino
        notas_reprobadas = notas_reprobadas + 1
        suma_reprobadas = suma_reprobadas + nota
    Fin_Si
Fin_Para
...
```

```
NOTA_MINIMA = 3.0
MINIMO_VALIDO = 0.0
MAXIMO_VALIDO = 5.0
notas_reprobadas = 0
notas_aprobadas = 0
suma_reprobadas = 0
suma_aprobadas = 0
```

```
N = int(input('Ingrese el número de estudiantes: '))
for i in range(N):
    nota = float(input('Ingrese la nota: '))
    if nota < MINIMO_VALIDO or nota > MAXIMO_VALIDO:
        print("ERROR: nota fuera de rango")
        continue
    if notas_validas == 0:
        nota_maxima = nota
        nota_minima = nota
    else:
        if nota > nota_maxima:
            nota_maxima = nota
        if nota < nota_minima:
            nota_minima = nota
    notas_validas = notas_validas + 1
    if nota >= NOTA_MINIMA:
        notas_aprobadas = notas_aprobadas + 1
        suma_aprobadas = suma_aprobadas + nota
    else:
        notas_reprobadas = notas_reprobadas + 1
        suma_reprobadas = suma_reprobadas + nota
...
```

Ciclos anidados – Forma general

- Al igual que las estructuras condicionales, es posible anidar ciclos.
- Los ciclos anidados son comúnmente empleados para recorrer tablas o matrices (lo veremos luego). Sin embargo, estas no son las únicas aplicaciones de estos. A continuación se listan algunos casos:
 - **Programas con menús:** un ciclo externo mantiene activo el menú hasta que el usuario decide finalizar la ejecución; dentro de una de las opciones puede existir un ciclo interno encargado de realizar una tarea repetitiva independiente (por ejemplo, leer y procesar los datos de N estudiantes antes de retornar al menú).
 - **Cálculo de series matemáticas:** la aproximación de funciones como $\sin(x)$ mediante una serie requiere un ciclo externo que recorra los términos de dicha serie, y un ciclo interno encargado de calcular el factorial necesario para cada término.
 - **Validación de datos con reintentos:** para cada dato que se requiere leer (ciclo externo), puede existir un ciclo interno que solicite repetidamente el valor hasta que este sea válido, antes de continuar con el siguiente dato.
 - **Recorrido de tablas o matrices:** constituye el caso más ilustrativo (una fila a la vez y, dentro de cada fila, cada columna), aunque no es el único escenario de aplicación.
- En todos los casos anteriores subyace la misma idea: una tarea que requiere repetirse en sí misma, ejecutada de manera reiterada por un proceso externo.

Ciclos anidados: Contextualización

- El ciclo interno está completamente contenido en el cuerpo del externo, con su propia inicialización, condición y actualización.
- Por cada iteración del externo, el interno se ejecuta en su totalidad; si el externo se repite F veces y el interno C veces, el cuerpo más interno se ejecuta $F \times C$ veces.
- Se usan variables de control distintas (por ejemplo, i y j) para cada nivel de repetición.
- A continuación se muestra la forma general empleando el ciclo **Para**:

Pseudocódigo

```
Para (i = inicio_ext, fin_ext, act_ext) Haga  
    Para (j = inicio_int, fin_int, act_int) Haga  
        ...  
    Fin_Para  
Fin_Para
```

Python

```
for i in range(inicio_ext, fin_ext):  
    for j in range(inicio_int, fin_int):  
        ...
```

Ciclos anidados: Ejercicios

- Cuadro de asteriscos: dado un entero positivo N, imprimir un cuadrado de asteriscos de NxN.
- Cuadro de asteriscos: dado un entero positivo N, imprimir un cuadrado de asteriscos de NxN el cual sea vacío por dentro. Es decir que el borde sea de solo un asterisco.
- Triángulo de asteriscos: dado un entero positivo N, imprimir un triángulo rectángulo de asteriscos de N filas, en el cual la fila i contiene i asteriscos.
- Hacer un programa que permita obtener los primeros N números primos.
- Dados un valor real x y un entero N, aproximar e^x mediante la serie:

$$e^x \approx \sum_{n=0}^N \left(\frac{x^n}{n!} \right) \approx 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots + \frac{x^n}{n!}$$

Referencias

1. <https://ellibrodepython.com/>
2. <https://www.w3resource.com/index.php>
3. <https://realpython.com/>
4. <https://curriculumresources.edu.gh/wp-content/uploads/2025/01/>
5. <https://python-course.eu/>
6. <https://realpython.com/cheatsheets/python/>
7. <https://realpython.com/python-conditional-statements/>
8. <https://realpython.com/ref/best-practices/conditionals/>
9. <https://makecode.microbit.org/courses/csintro>
10. <https://makecode.microbit.org/courses>
11. <https://realpython.com/python-while-loop/>
12. <https://realpython.com/ref/keywords/while/>

Eso es todo amigos

